

# Space-efficient graph coarsening with applications to succinct planar encodings

Nina Hammer<sup>1</sup>, Frank Kammer<sup>1</sup>, Johannes Meintrap<sup>1,\*</sup>

THM, University of Applied Sciences Mittelhessen, Giessen, Germany

## ARTICLE INFO

Editor: Dr. Pinyan Lu

### Keywords:

Planar graph  
H-minor-free  
Space-efficient  
Separator  
Tree decomposition

## ABSTRACT

We present a novel space-efficient graph coarsening technique for  $n$ -vertex planar graphs  $G$ , called *cloud partition*, which partitions the vertices  $V(G)$  into disjoint sets  $C$  of size  $O(\log n)$  such that each  $C$  induces a connected subgraph of  $G$ . Using this partition  $\mathcal{P}$  we construct a so-called *structure-maintaining minor*  $F$  of  $G$  via specific contractions within the disjoint sets such that  $F$  has  $O(n/\log n)$  vertices. The combination of  $(F, \mathcal{P})$  is referred to as a *cloud decomposition*.

For planar graphs we show that a cloud decomposition can be constructed in  $O(n)$  time and using  $O(n)$  bits. Given a cloud decomposition  $(F, \mathcal{P})$  constructed for a planar graph  $G$  we are able to find a balanced separator of  $G$  in  $O(n/\log n)$  time. Contrary to related publications, we do not make use of an embedding of the planar input graph. We generalize our cloud decomposition from planar graphs to  $H$ -minor-free graphs for any fixed graph  $H$ . This allows us to construct the succinct encoding scheme for  $H$ -minor-free graphs due to Blelloch and Farzan (CPM 2010) in  $O(n)$  time and  $O(n)$  bits improving both runtime and space by a factor of  $\Theta(\log n)$ .

As an additional application of our cloud decomposition we show that, for  $H$ -minor-free graphs, a tree decomposition of width  $O(n^{1/2+\epsilon})$  for any  $\epsilon > 0$  can be constructed in  $O(n)$  bits and a time linear in the size of the tree decomposition. A similar result by Izumi and Otachi (ICALP 2020) constructs a tree decomposition of width  $O(k\sqrt{n}\log n)$  for graphs of treewidth  $k \leq \sqrt{n}$  in sublinear space and polynomial time.

## 1. Introduction

Graphs are used to model a multitude of systems that can be expressed via entities and relationships between these entities. Many real-world problems operate on very large graphs for which standard algorithms and data structures use too much space. This has spawned an area of research with the aim of reducing the required space. Examples include large road-networks [1] or social graphs arising from interactions between users of large internet communities [2]. Once the graphs become too massive, it is often infeasible to run standard in-memory algorithms using standard data structures and approaches. Commonly, one must resort to streaming algorithms in such cases, with the caveat of lower solutions quality and possible increases in runtime [3]. Therefore, it is of high interest to find compact representations of graphs, space-efficient algorithms and other space-efficient or succinct data structures, such that even larger graphs can be handled in-memory. In the following we denote by  $n$  the number of vertices of a graph under consideration and  $m$  the number of edges. An algorithm is called space-efficient if it has (almost) the same asymptotic

\* Corresponding author.

E-mail addresses: [nina.hammer@mni.thm.de](mailto:nina.hammer@mni.thm.de) (N. Hammer), [frank.kammer@mni.thm.de](mailto:frank.kammer@mni.thm.de) (F. Kammer), [johannes.meintrup@mni.thm.de](mailto:johannes.meintrup@mni.thm.de) (J. Meintrap).

<sup>1</sup> Funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) 379157101.

runtime as a standard algorithm for the same problem, but uses asymptotically fewer bits. Examples include space-efficient graph searching algorithms such as depth-first search and breadth-first search, which run in linear time, but use  $O(n \log n)$  bits with standard methods. Space-efficient solutions lower the space requirement to  $O(n)$  bits and keep the runtime asymptotically (almost) the same. For a data structure or algorithm to be called *succinct* the space used must be  $Z + o(Z)$  bits with  $Z$  being the information theoretic minimum to store the data. One of the most researched topics regarding space-efficiency are graph-traversal algorithms such as depth-first search (DFS). It is still an open research question if a linear-time DFS exists that uses  $O(n)$  bits, with the current best bound of  $O(n + m + \min\{n, m\} \log^* n)$  due to Hagerup [4], with  $\log^* n$  being the iterated logarithm. This result is the work of gradual improvements over the span of seven years spanning multiple results by multiple research groups, with some of the first results providing a runtime  $O(m \log n)$  time due to Asano et al. [5] and a runtime of  $O((n + m) \log \log n)$  due to Elmasry et al. [6]. Other typical problems that can be easily solved in settings where space is of no concern, such as storing spanning trees or simple mappings between vertices, require new problem-specific strategies when considered in the space-efficient setting.

Another interesting setting is aiming for arbitrary polynomial time runtime, but using only  $o(n)$ , or even  $O(\log n)$  bits. In such settings information can be recomputed in polynomial time, and for problems such as mappings this is often trivial, while other problems such as deciding if two vertices are connected by a path in a graph requires the very involved algorithm due to Reingold [7], but is quite easy in a space-efficient setting. Thus, our space-efficient setting differentiates itself quite strongly from both, the well-researched sublinear-space settings and the common settings that do not regard space as limited.

Many large graphs that arise in practice have some known structural property that allows the design of specialized techniques that make use of these properties. A common such structural property is the existence of small separators. Such a *separator* is a small subset of vertices whose removal disconnects the graph if it was connected previously, or increases the number of connected components if it was not connected. For the graphs of interest to this work the separators are so-called balanced separators. For now the intuition suffices that such balanced separators split the graph in somewhat equally parts. Section 3 contains precise definitions of these terms and also defines what we consider small in regard to separators. Arguably the most well-known graphs that contain such small balanced separators are planar graphs, which are graphs that can be drawn in the plane without overlapping edges. All planar graphs contain a balanced separator of size  $O(\sqrt{n})$  [8]. Similar separator theorems exist for almost-planar graphs [9] (which includes road networks) and well-formed meshes such as nearest neighbor graphs [10] and  $H$ -minor-free graphs for some fixed graph  $H$  [11]. For arbitrary separable graphs there exists a polylogarithmic approximation algorithm for finding balanced separators due to Leighton and Rao [12].

We present a novel partitioning scheme called *cloud partition* for planar graphs that partitions the vertices of a connected input graph  $G$  into connected subsets called *clouds* that induce a connected subgraph and are of size  $O(\log n)$ . For a cloud partition  $\mathcal{P}$  constructed for  $G$  we construct a so-called *structure-maintaining minor*  $F$  of  $G$  induced by  $\mathcal{P}$ . For easier reading comprehension we call vertices of such a minor *nodes*. Intuitively, a node  $v \in V(F)$  is mapped to one or more clouds  $C \in \mathcal{P}$  such that  $|V(F)| = O(n / \log n)$ —the exact mapping is outlined in Section 4. A solution for some problems such as finding small separators can be found in  $F$  and then translated to an approximate solution for  $G$ . As  $F$  contains  $O(n / \log n)$  nodes, the time and space bounds of linear or superlinear algorithms can be decreased by a factor of  $\Omega(\log n)$  when being executed on  $F$  instead of  $G$ . We show how this speedup is especially helpful for recursive algorithms such as the recursive separator search used during the computation of the succinct representation of separable graphs due to Blelloch and Farzan [13]. Additionally, we show that small modifications of this recursive separator search can be used to find a tree decomposition of width  $O(n^{1/2+\epsilon})$  for any  $\epsilon > 0$  for planar graphs in  $O(n)$  bits and a time linear in the size of the tree decomposition. Finally, we generalize our partitioning scheme from planar graphs to  $H$ -minor-free graphs for any fixed graph  $H$ .

One of the key points of our novel scheme is that we do not make use of a (planar) embedding of the input graph, as there is no known way to construct such an embedding with  $O(n)$  bits and in linear time. It is often implied that an embedding is given alongside a planar graph in many of the works regarding planar graphs, as it is computable in linear time [14] when space is of no concern. Additionally, it is often required that the planar graph is maximal. Both of these properties are, for example, required for the major result of the  $O(\sqrt{n})$ -separator theorem [8] or the so-called  $r$ -partition of Klein et al. [15] (effectively a recursive application of the aforementioned separator theorem). Again, if the graph is not maximal, it can be easily made maximal in linear time with the help of an embedding. In a space-efficient context we can not make use of either of these properties and are thus quite limited. For sublinear space settings there exist algorithms that produce an embedding in polynomial time with a rather large polynomial degree [16,17]. Thus, the  $O(n)$ -bit setting provides a unique challenge due to the additional goal of matching the runtime of non space-efficient algorithms.

Succinct and space-efficient representations of planar graphs is a highly researched topic partially due to the practical applications. For compressing planar graphs without regards to providing fast access operations refer to the work of Keeler et al. [18] for an  $O(n)$  bits representation and for a compression within the information theoretic lower bound refer to He et al. [19]. Due to Munro and Raman [20] there exists an encoding using  $O(n)$  bits that allows constant-time queries which has subsequently been improved by Chiang et al. [21] to use a constant factor less space. We use the succinct representation due to Blelloch and Farzan, which allows encoding arbitrary separable graphs and subsequently allows constant-time adjacency-, neighborhood- and degree-queries [13], which builds on the work of Blanford et al. [22]. For  $H$ -minor-free graphs for a fixed graph  $H$ , their encoding takes  $\Theta(n \log n)$  time and  $\Theta(n \log n)$  bits using their described technique of recursive separator searches. We are able to improve it to  $O(n)$  time and  $O(n)$  bits. For planar graphs we only improve the space-requirement from  $\Theta(n \log n)$  to  $O(n)$  due to the algorithm of Goodrich [23] which can be used instead of the generic recursive separator search described by Blelloch and Farzan. Note that all mentioned works above regarding succinct representations of planar graphs use  $\Theta(n \log n)$  bits during the construction. Additionally, they assume a planar embedding is given explicitly or implicitly by making direct use of it, or referring to other results as sub-routines that require them.

For maximal planar graphs there exist special encodings. Some major research in this field is due to Aleardi et al. [24–27], which deals with practical and theoretical representations of such maximal planar graphs, typically arising in applications making use of geometric meshes or computer graphics. Note that these space-efficient encodings of maximal planar graphs also make use of and store an embedding, as navigation of the faces is a vital feature in these applications.

The general technique of graph coarsening has been heavily used in the past for practical and theoretical algorithms [28–32]. A typical approach is to find a matching  $E' \subset E(G)$  for a graph  $G$  and contract the edges in  $E'$  to find a minor  $F$  of  $G$ . The problem at hand is then solved on  $F$  and the solution is translated to an approximate solution for  $G$  [30]. Highly specialized practical algorithms using sophisticated data structures are implemented in the METIS [33] and SCOTCH [34] library. Other approaches such as graph coarsening on the GPU [35,36] or graph coarsening via neural networks [37] have been developed. Note that the use of sophisticated data structures that focus on runtime optimization do not lend themselves to modifications to make them space-efficient. Thus, there was a need to develop a novel partitioning scheme for this work, in particular a scheme that does not require an embedding when dealing with planar graphs.

For finding tree decompositions in a space-efficient manner refer to the work of Kammer et al. [38] and the work of Izumi and Otachi [39]. Izumi and Otachi showed that for a given graph  $G$  with treewidth  $k \leq \sqrt{n}$  there exists an algorithm that obtains a tree decomposition of width  $O(k\sqrt{n} \log n)$  in polynomial time and  $O(k\sqrt{n} \log^2 n)$  bits. Note that the polynomial degree in the runtime is rather large due to the use of the well known  $s - t$  reachability algorithm of Reingold [7]. Izumi and Otachi additionally mention that for planar graphs there exists a polynomial-time and sublinear-space algorithm for finding a tree decomposition of a planar graph due to a simple recursive separator search with the result of Imai et al. [40], which present an algorithm for finding balanced separators of size  $O(\sqrt{n})$  in planar graphs with sublinear space and polynomial runtime. Note that the algorithm of Imai et al. makes use of an embedding of the input graph and also uses the algorithm of Reingold. In contrast, we present an algorithm that computes a tree decomposition of width  $O(n^{1/2+\epsilon})$  in time linear in the size of the tree decomposition using  $O(n)$  bits. As the tree decomposition has  $O(n)$  bags of size  $O(n^{1/2+\epsilon})$  this results in a runtime of  $O(n^{3/2+\epsilon})$ .

First, in Section 2 we give a high-level description of our coarsening scheme. Following that, in Section 3 we present concepts needed to understand the subsequent sections. In Section 4 we present our space-efficient graph-coarsening framework. Following that, in Sections 5 and 6 we show how this scheme is used for finding separators, tree decompositions and constructing succinct encodings of planar graphs. In Section 7 we generalize our work to  $H$ -minor-free graphs.

## 2. Conceptual breakdown

In this section, we give an intuitive overview of our results. We sketch the broad ideas, ignoring some of the technical details and edge cases. We introduce some definitions that will be precisely defined in the next sections, for now intuitions suffice. Let  $G = (V, E)$  be a planar graph. Our goal is to coarsen the graph  $G$ , i.e., we want to shrink  $G$  in such a way that the basic structure of  $G$  is preserved, but the number of vertices and edges is significantly reduced. Once we have obtained this coarsened graph, we can solve various problems by solving them on the coarsened graph, and expanding the solution to obtain an approximate solution for the given problem on the original graph. In particular, we want to shrink  $G$  to a minor  $F$  of  $G$  that contains  $\Theta(n/k)$  vertices. Roughly speaking, we want every vertex of  $F$  to represent  $\Theta(k)$  (adjacent) vertices of  $G$ . We will not be able to exactly achieve this result, but intuitively speaking, this would be our ideal scenario. In the following we describe a simple algorithm for coarsening  $G$ , and discuss (1) why it does not achieve our desired results, and (2) why the problems that arise are based on structurally simple cases that can easily be handled separately. Our coarsening algorithm is based on repeatedly performing a BFS until  $k$  (previously unvisited) vertices are visited. Once  $k$  vertices  $P \subseteq V$  are visited, or no unvisited vertices can be reached, we stop the BFS. We then repeat this procedure at a new (unvisited) vertex. Once no unvisited vertices remain, all so constructed sets  $P$  form a partition  $\mathcal{P}$  of the vertices of  $G$  such that each  $P \in \mathcal{P}$  induces a connected subgraph of  $G$ . We call each  $P \in \mathcal{P}$  a *cloud*, and  $\mathcal{P}$  a *cloud partition*. See Algorithm 1 for the pseudo code of this partitioning algorithm.

**Data:** Graph  $G = (V, E)$ , cloud size  $k$

**Result:** Cloud partition  $\mathcal{P}$

$\mathcal{P} \leftarrow \emptyset$

$U \leftarrow V$

**while**  $U \neq \emptyset$  **do**

    Let  $u$  be any vertex from  $U$

$P \leftarrow \text{BFS}(G, u, U, k)$

$\mathcal{P} \leftarrow \mathcal{P} \cup \{P\}$

$U \leftarrow U \setminus P$

**end**

**return**  $\mathcal{P}$

**Algorithm 1:** Partition a graph  $G$  into clouds of size at most  $k$ . The sub-routine  $\text{BFS}(G, u, U, k)$  refers to a breadth-first search of a graph  $G$  starting at vertex  $u$  that only traverses vertices of  $U$ . Once the BFS has either visited  $k$  vertices, or no more vertices can be reached, the set of visited vertices is returned.

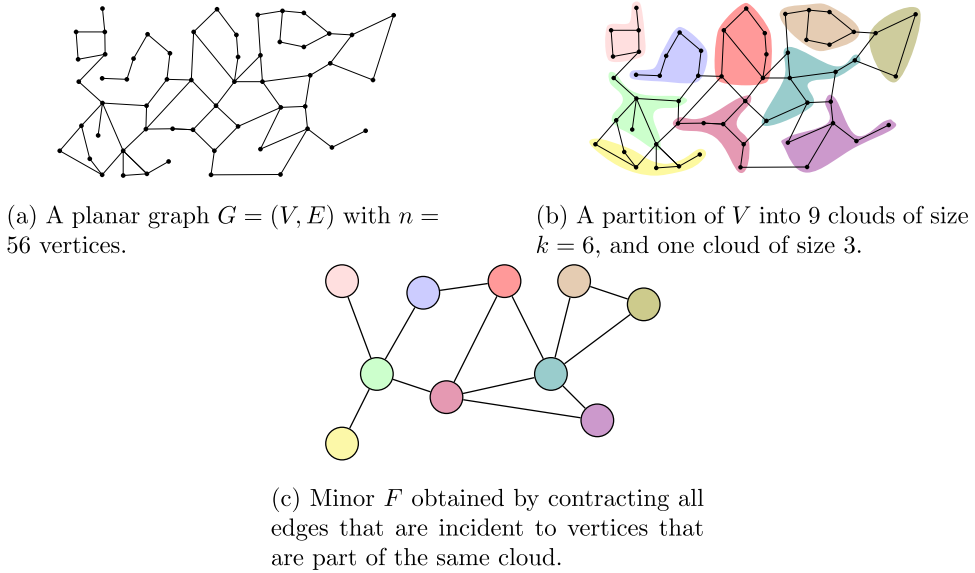
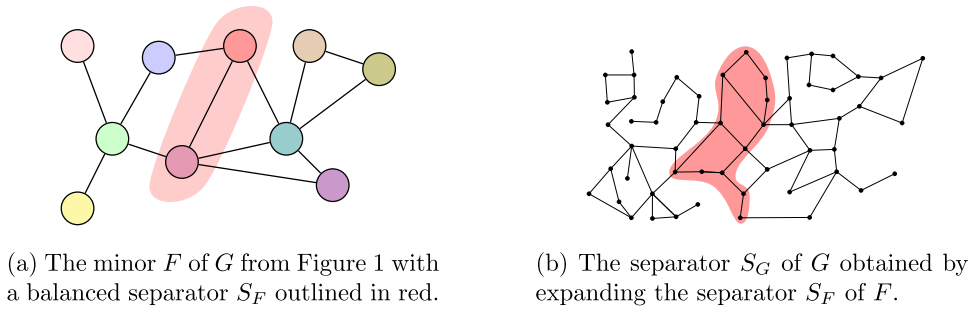


Fig. 1. Sketch of the graph coarsening scheme.

Fig. 2. A separator  $S_F$  of the minor  $F$  of  $G$  can be translated to a separator  $S_G$  of  $G$ .

What we want is for this simple algorithm to partition the vertices into  $\Theta(n/k)$  clouds, with all clouds (except possibly one) of size  $\Theta(k)$ . For now, let us assume that the simple algorithm does exactly what we want. To get a coarsened graph, we then merge all vertices that are part of the same cloud  $P \in \mathcal{P}$  into a single vertex. The resulting graph—let us call it  $F$ —is a minor of  $G$ . To make it easier to distinguish between  $G$  and  $F$ , we refer to the vertices of  $F$  as nodes. The graph  $F$  is our desired coarsened graph with  $\Theta(n/k)$  nodes such that each node of  $F$  represents a cloud of  $G$ . We refer to  $F$  as a *structure-maintaining minor*. See Fig. 1 for a visual example.

Now consider the problem of finding a balanced separator of  $G$ . First, we find a balanced separator  $S_F$  of  $F$  of size  $O(\sqrt{|V(F)|})$  using the standard separator theorem for planar graphs. By expanding each node of  $S_F$  to the cloud it represents we get a balanced separator  $S_G$  of  $G$ . Fig. 2 visualizes the idea. Unfortunately, there is a problem with this approach. The simple BFS-based algorithm is not guaranteed to find a vertex partition with our desired properties. Worse, there are (planar) graphs where no partition into  $\Theta(n/k)$  clouds exists for some non-trivial values  $k$ . Fig. 3 shows two such counter examples.

We call clouds of size less than  $k$  *small clouds*, and we call two clouds *adjacent* if there is an edge that has an endpoint in each of the clouds. In the two example graphs  $G = (V, E)$  shown in Fig. 3 there are  $\Theta(n)$  small clouds for every possible vertex partition  $\mathcal{P}$  of  $V$  such that each  $P \in \mathcal{P}$  induces a connected graph. It turns out that for planar graphs, there are only two types of small clouds that are problematic for our use case. First, so-called *leaf clouds*, that are clouds that are adjacent to only one other cloud, and second, so-called *bridge clouds*, that are adjacent to two other clouds. Leaf clouds appear in the graph of Fig. 3(a), and bridge clouds appear in the graph of Fig. 3(b). As the number of leaf and bridge clouds is unbounded, our coarsening algorithm can not produce our desired coarsened graph  $F$ . Recall that ideally we want  $F$  to contain  $\Theta(n/k)$  vertices such that each vertex of  $F$  represents  $\Theta(k)$  vertices of  $G$ . Later we show how to deal with this problem. Intuitively, leaf and bridge clouds adhere to very simple structures that can be handled as special cases. In particular, we combine multiple leaf clouds adjacent to a single other cloud  $C$  to a so-called *meta leaf cloud*, and analogously multiple bridge clouds adjacent to two clouds  $C, C'$  to so-called *meta bridge clouds*. When replacing leaf and bridge-clouds with these meta clouds, we can show that  $F$  contains our desired amount of vertices. We then handle vertices of  $F$  that represent a meta cloud as special cases. Note that there may be small clouds adjacent to  $\geq 3$  other clouds (called *critical clouds*), but

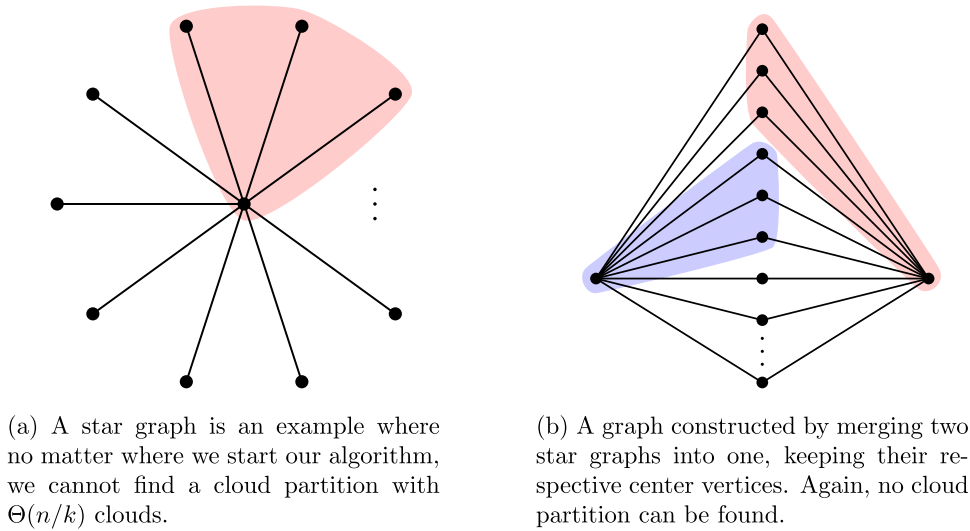


Fig. 3. Examples of graphs where no cloud partition exists. In both examples  $k$  is set to 4.

due to planarity we can show that the number of such clouds is bounded by  $O(n/k)$ , so asymptotically they are not relevant to us. The details are given in the next sections.

Since our goal is to implement the sketched framework for graph coarsening in a space-efficient way, we need to be careful about the technical details of how the framework is implemented. In particular, we use  $k = c \log n + 1$ , where  $c$  is an arbitrary constant, and our goal is to construct the structure-preserving minor  $F$  in time  $O(n)$  while using only  $O(n)$  bits. One of the main problems we have to overcome is to store a bidirectional mapping between nodes of  $F$  and clouds (i.e., sets of vertices) of  $G$ . Roughly speaking, for each node  $w$  of  $F$ , we store a single vertex of  $G$ , from which we can reconstruct the cloud represented by  $w$ . This reconstruction step is performed by executing a BFS on a special space-efficient subgraph  $G'$  of  $G$  that contains only edges connecting vertices that are part of the same cloud. In particular, we store for each vertex  $w$  of  $F$  a so-called *representative*  $u \in V(G)$ , such that  $u$  is contained in the cloud  $C$  represented by vertex  $w$  of  $F$ . To output the vertices of  $C$ , we execute a BFS in the aforementioned graph  $G'$ , starting at vertex  $u$ . For clouds that are not leaf or bridge clouds, it is sufficient as we have  $O(n/k) = O(n/\log n)$  such clouds, and can therefore afford to store the representative explicitly using  $\Theta(\log n)$  bits per entry. As we have only a trivial bound of  $O(n)$  for bridge and leaf clouds, we must use a different approach for storing the representative for these clouds, as otherwise we require  $\omega(n)$  bits. Intuitively, we solve this problem by introducing the previously mentioned meta clouds, and storing one representative per meta cloud. Recall that each such meta-cloud represents a set of bridge or leaf clouds with the property that all clouds  $C_1, \dots, C_k$  represented by a meta cloud  $C$  are adjacent to the same cloud  $C'$  if they are leaf clouds, and the same clouds  $C'$  and  $C''$  if they are bridge clouds. One caveat of this approach is that meta clouds are not connected, and as such we are unable to run a BFS to obtain all vertices of a meta-cloud in the graph  $G'$ . A technical problem we face in the next sections.

### 3. Background

We operate in the standard word RAM model of computation with word size  $w = \Omega(\log n)$ . This assumes the existence of read-only input memory, read/write working memory and write-only output memory. When we talk about space-usage we focus on bits used in the read/write working memory. Sometimes this model is also referred to as the *register input model* [41]. This is a common setting for space-efficient algorithms.

We make use of common graph theoretic notations and terminology. Refer to Diestel [42] for more information. When using space-efficient graph algorithms the exact representation of the input graph is important, in particular the common adjacency list interface is not sufficient [5,6]. Instead, we assume that any input graph is given via *adjacency arrays* or an equivalent interface. Given a vertex  $u$  and index  $i \leq \text{degree}(u)$  this allows us to determine in constant time the  $i$ th edge  $\{u, v\}$  out of  $u$ . All our input graphs are assumed to be undirected with the vertices labeled from  $1, \dots, n$ . Such a labeling is given implicitly by the order of the adjacency arrays. Given the use of adjacency arrays we store each undirected edge  $\{u, v\}$  as two directed *arcs*  $uv$  and  $vu$ . For each arc  $uv$  we call  $vu$  the *co-arc* of  $uv$ . We assume the existence of so-called *crosspointers* that allow us to find the co-arc of an arc  $uv$  in constant time. Typically, this is realized by storing an index  $i$  in addition to the vertex  $v$  in  $u$ 's adjacency array. The index  $i$  then indicates the position of  $u$  in  $v$ 's adjacency array. We assume w.l.o.g. that every given graph  $G$  is connected as otherwise all our techniques can be done iteratively for each connected component of  $G$ . We assume that a given input graph is in read-only memory.

A *separator*  $S$  of a graph  $G = (V, E)$  is a subset of  $V$  such that its removal from  $V$  divides  $V$  into non-empty sets  $A \subset V$  and  $B \subset V$  so that  $\{A, S, B\}$  is a partition of  $V$  with the constraint that all paths from a vertex  $u \in A$  to a vertex  $v \in B$  contain at least one vertex of  $S$ . If  $|A| < \alpha n$  and  $|B| < \alpha n$  for some  $\alpha < 1$ , then  $S$  is called a  $\alpha$ -balanced separator or simply *balanced separator*.

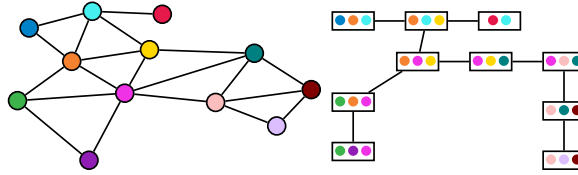


Fig. 4. A graph and its tree decomposition. This graph has treewidth 2.

A family of graphs  $\mathcal{G}$  that is closed under taking vertex-induced subgraphs satisfies the  $f(\cdot)$ -separator theorem exactly if, for constants  $\alpha < 1$  and  $\beta > 0$ , each member  $G \in \mathcal{G}$  has an  $\alpha$ -separator  $S$  of size  $|S| < \beta f(n)$ . We say a family of graphs  $\mathcal{G}$  is *separable* exactly if it satisfies the  $n^c$ -separator theorem for some  $c < 1$ . We say a graph is separable if it belongs to a separable family of graphs. For planar graphs there exist an  $O(\sqrt{n})$ -separator theorem with  $\alpha = 2/3$  that runs in linear time [8], later extended to graphs of bound genus with the same runtime [43]. For minor-closed graph classes excluding the complete graph  $K_t$  on  $t$  vertices for some constant  $t$  there exists an  $O(n^{(2-\epsilon)/3})$ -separator theorem with runtime  $O(n^{1+\epsilon} + m)$  [11]. Note that this includes  $H$ -minor-free graphs for any fixed graph  $H$ .

The definition of the separable graph families are not closed under taking minors, but vertex-induced subgraphs [13]. As we specifically construct minors  $F$  of a separable graph  $G$  at various points in this work, we require the more restrictive property that  $F$  also belongs to the same family of separable graphs as  $G$ , which holds for all graph classes explicitly mentioned in the previous paragraph.

Due to Lipton et al. [44] it is known that a separable graph class  $\mathcal{G}$  has *bound density*  $d$  for some constant  $d$ . This means that each  $G \in \mathcal{G}$  contains at most  $dn$  edges. In particular, we use the well-known fact, by Euler's Formula, that for a planar graph  $G = (V, E)$  it holds that  $|E| \leq 3|V| - 6$ . For planar bipartite graphs a stronger bound  $|E| \leq 2|V| - 4$  holds.

A *tree decomposition* of a graph  $G = (V, E)$  consists of a tree  $T$  and a family  $\mathcal{X}$  of subsets  $X_w$  (called *bags*) of  $V$ , one for each  $w \in V(T)$ , such that:

1.  $\bigcup_{w \in V(T)} X_w = V$
2. for all  $\{u, v\} \in E$  there exists  $w \in V(T)$  such that  $u, v \in X_w$
3. for all  $w_1, w_2, w_3 \in V(T)$ , if there is a path from  $w_1$  to  $w_3$  that contains  $w_2$ , then  $X_{w_1} \cap X_{w_3} \subseteq X_{w_2}$ .

The *width* of a tree decomposition is the size of the largest bag minus one. The *treewidth* of a graph  $G$  is the smallest width amongst all possible tree decompositions of  $G$ . See Fig. 4 for an example.

We make use of *indexable dictionaries*, which is a structure that supports constant-time rank-select queries over a bitvector. The  $\text{rank}(i)$  operation counts the number of occurrences of bits set to 1 before the  $i$ th index and the  $\text{select}(i)$  operation returns the index of the  $i$ th bit set to 1.

**Lemma 1** ([45]). *Given a bitvector  $S$  of length  $\ell$  there is an indexable dictionary on  $S$  that requires  $o(\ell)$  additional bits, supports rank-select queries in constant time and can be constructed in  $O(\ell)$  time.*

What can be thought of as a dynamic alternative to indexable dictionaries is the so-called *choice dictionary* [46]. A choice dictionary is initialized for a universe  $1, \dots, \ell$  and supports constant time *insert*, *delete*, and *contains* operations, with the latter returning *true* exactly if an element of the universe is contained in the choice dictionary. Additionally, the choice dictionary offers the operation *choice* that returns an arbitrary member and iteration over its members. The iteration outputs all members of a choice dictionary in constant time per member. All operations run in constant time and the iteration over its members is linear in the number of members. The following lemma has been adapted from Kammer and Sajenko [47], with some minor rephrasing to make it more clear in the context of this paper.

**Lemma 2.** *There is a succinct choice dictionary initialized for the universe  $1, \dots, \ell$  that occupies  $\ell + o(\ell)$  bits and provides constant-time *insert*, *delete*, *contains* and *choice* operations and constant-time (per member) iteration. The choice dictionary can be initialized in  $O(\ell)$  time.*

We make use of a folklore technique called *static space allocation* allowing us to store  $\ell$  items of varying size compactly. The following description is adapted from Kammer et al. [48]. Each item  $B_k$  occupies  $d_k$  bits for  $k \in \{1, \dots, \ell\}$ . Denote by  $L$  the amount of bits all these items totally occupy. We want to store all these items with  $L + o(L)$  bits such that we can access each item  $B_k$  in  $O(1)$  time for  $k \in \{1, \dots, \ell\}$ . In  $O(\ell + L)$  time compute the sums  $s_k = k + \sum_{j=1}^{k-1} d_j$  and an indexable dictionary over a bitvector  $S$  of size  $\ell + L$  with  $S[i] = 1$  exactly if  $i = s_k$  for  $k \in \{1, \dots, \ell\}$ . The location of  $B_k$  is then equivalent to  $\text{select}_S(s_k) - s_k$ . Refer to [48–50] for more information and detailed descriptions.

For traversing graphs we use standard breadth-first search (BFS), which puts a start vertex in a first queue, then the unprocessed neighbors of the first queue in a second queue, swaps the queues and repeats. It can easily be seen that space requirement is  $O(n \log n)$  bits. More precise, the BFS uses  $O(\ell \log n)$  bits where  $\ell$  is the maximum number of vertices in a queue at any point of the BFS.

In this work we need to store subgraphs  $G'$  of a given graph  $G = (V, E)$ . Similar to the techniques used by Hagerup et al. [50] we are able to store  $G'$  using  $O(n + m)$  bits. We store a subgraph  $G'$  of  $G$  via  $n$  choice dictionaries  $D_v$  for each vertex  $v \in V(G)$ . Each  $D_v$  has length  $d_v$  with  $d_v$  being the degree of  $v$  in  $G$ . The choice dictionaries are stored using static space allocation. A member  $i$  in  $D_v$  then indicates the existence of the  $i$ th arc out of  $v$ 's adjacency array. Another choice dictionary  $D'$  of length  $n$  can be used to mark

vertices of degree  $> 0$ . Iterating over the neighborhood of a vertex  $v \in V(G')$  can be done in linear time of the degree of  $v$  in  $G'$ . This additionally allows dynamic insertion and deletion of edges as long as  $G'$  remains a subgraph of  $G$ , i.e., no new edges can be added. Note that we do not directly allow the deletion of vertices. Instead, when a vertex has degree 0 in  $G'$  (due to the deletion of all incident edges) it is marked in  $D'$ . This allows to iterate over all vertices  $V' = \{v \in V(G') \mid d_v > 0\}$  in  $O(|V'|)$  time with  $d_v$  being the degree of  $v$  in  $G'$ . Additionally, an arbitrary vertex  $v \in V(G')$  with  $d_v > 0$  can be obtained via  $D'.choice()$ . We refer to such a structure as a *dynamic subgraph*  $G'$  of  $G$ . Note that a dynamic subgraph  $G'$  can be used to direct an edge  $\{u, v\} \in E(G')$  by deleting only the arc  $uv$  or  $vu$  in  $G'$ . Using this we are able to implement the next lemma in  $O(n)$  time and  $O(n)$  bits using the same algorithm as described in Blandford et al. [22].

**Lemma 3.** *Let  $G$  be a separable graph. We can obtain the directed graph  $G'$  of  $G$  such that each vertex of  $G'$  has bounded in-degree (out-degree) in  $O(n)$  time and  $O(n)$  bits.*

**Proof.** We implement a space-efficient algorithm of the algorithm outlined by Blandford et al. [22]. Note that for all separable graph classes there exist some constant  $b$  such that at least half of the vertices have degree  $\leq 2b$  [22] and each separable graph  $G = (V, E)$  has  $E = O(|V|)$  [44]. The algorithm due to Blandford et al. takes as input a separable graph  $G = (V, E)$ , obtains the vertices  $V' \subset V$  such that each  $v \in V'$  has degree  $\leq 2b$ . All edges of  $E$  with an endpoint in  $V'$  are then directed towards  $V'$ . Then all vertices of  $V'$  are removed, and the process is repeated until all edges are handled this way. The runtime is  $O(n)$  as  $> 1/2|V|$  vertices are removed in each step. We now describe the technical details of how to obtain such a directed graph in  $O(n)$  time and  $O(n)$  bits.

Let  $G$  be a separable graph and  $G_h, G'$  dynamic subgraphs of  $G$ , both initially containing all edges of  $G$ . Note that each undirected edge  $\{u, v\} \in E$  is represented by two arcs  $uv$  and  $vu$ . Removing one of these arcs can then be thought of as directing this undirected edge from  $u$  to  $v$  or  $v$  to  $u$ . In the following we construct a directed version of  $G$  iteratively in  $G'$ . We use  $G_h$  as a helping structure during the construction. As an additional structure we use a choice dictionary  $Q$  which functions as a queue for vertices of degree  $\leq 2b$ . Iterate over all vertices  $v$  of degree  $> 0$  in  $G_h$  using the choice dictionary  $D'$  (Section 3). For each  $v$  obtain the degree  $d_v$  of  $v$  by iterating over the adjacency list of  $v$  in  $G_h$ . If  $d_v \leq 2b$ , then add  $v$  to  $Q$ . Finding and adding all these vertices to  $Q$  takes  $O(n + m) = O(n)$  time. In  $G'$  remove all arcs  $uv$  with  $u \in Q$  and  $v \neq Q$ . In the case that both  $u$  and  $v$  are in  $Q$  remove the arc  $uv$  with  $u < v$ . Removing all these arcs in  $O(n)$  time. For each  $v \in Q$  remove all incident edges from  $G_h$  and continue the iteration in  $G_h$  until all edges are handled. The runtime is the same as due to Blandford et al. while the use of dynamic subgraphs uses  $O(n)$  bits.  $\square$

#### 4. Graph coarsening framework for planar graphs

In this section we outline a strategy for coarsening a planar graph  $G = (V, E)$ . The idea is to create a specific type of partition  $\mathcal{P}$  of  $V$ , which we call *cloud partition* that induces a unique minor  $F$  of  $G$ , defined later. We refer to each  $C \in \mathcal{P}$  as a *cloud*. Thus, a cloud is a set of vertices.

**Cloud partition.** In the following we describe the exact specifications that define a cloud partition. For each  $C \in \mathcal{P}$ ,  $C$  induces a connected subgraph of  $G$  and  $|C| \leq \lceil c \log n \rceil + 1$  for an arbitrary, but fixed constant  $c$ . We differentiate between different types of clouds  $C \in \mathcal{P}$ , which we define after introducing some terminology. Let  $C_1, C_2 \in \mathcal{P}$  with  $C_1 \neq C_2$ . We call an edge  $\{u, v\} \in E$  a *border edge* exactly if  $u \in C_1$  and  $v \in C_2$ . We then refer to  $C_1$  and  $C_2$  as *adjacent* or *neighbors* and as *incident* to  $\{u, v\}$ . Furthermore, we call  $C$  a *big cloud* if  $|C| = \lceil c \log n \rceil + 1$  and a *small cloud* otherwise. We call a small cloud  $C$  a *leaf cloud* if  $C$  is adjacent to one cloud, a *bridge cloud* if it is adjacent to two clouds and a *critical cloud* if it is adjacent to at least three clouds. We call two clouds  $C_1, C_2$  adjacent to a bridge cloud  $B$  *connected by  $B$* . A cloud partition is created by Algorithm 1, i.e., by repeatedly running a BFS that collects at most  $\lceil c \log n \rceil + 1$  vertices, with each iteration producing a new cloud. Only a partition created by Algorithm 1 is referred to as a cloud partition. By fixing the search for unvisited vertices and the BFS algorithm, each graph has a fixed cloud partition. The next two observations are derived directly from the process of creating a cloud partition.

**Observation 1.** *Let  $\mathcal{P}$  be an arbitrary cloud partition created for a graph  $G$ . Then no two small clouds  $C_1, C_2 \in \mathcal{P}$  are adjacent.*

**Observation 2.** *Let  $\mathcal{P}$  be an arbitrary cloud partition created for a graph  $G$ . Since each big cloud has size  $\lceil c \log n \rceil + 1$  for a constant  $c$ ,  $\mathcal{P}$  contains at most  $n/(\lceil c \log n \rceil + 1)$  big clouds.*

For planar graphs, the number of critical clouds can be bounded similarly. The proof is based on the fact that a planar graph has  $O(n)$  edges.

**Lemma 4.** *Let  $\mathcal{P}$  be a cloud partition created for a planar graph  $G$  containing  $k$  big clouds. Then  $\mathcal{P}$  contains  $O(k)$  critical clouds.*

**Proof.** Consider the graph  $F = (V', E')$  constructed by contracting the vertices of each cloud  $C \in \mathcal{P}$  to a single vertex  $v \in C$ . The graph  $F$  is a minor of  $G$  with  $|V'| = |\mathcal{P}|$  and each vertex  $v \in V'$  represents a single cloud  $C_v \in \mathcal{P}$ . Two vertices  $v, u \in V'$  are adjacent in  $F$  exactly if  $C_v$  and  $C_u$  are adjacent. For brevity's sake we refer to vertices of  $F$  introduced for critical clouds as critical vertices, and vertices introduced for big clouds as big vertices. Assume that all vertices that are not big vertices or critical vertices are removed in  $F$  and all edges between big vertices are removed as well. This turns  $F$  into a bipartite planar graph due to Observation 1, with big vertices on one side of the bipartition, and small and critical on the other. It is well-known that  $|E'| \leq 2|V'| - 4$  for bipartite planar graphs. Denote by  $\ell$  the number of critical vertices and  $k$  the number of big vertices in  $F$ . Note that for each critical vertex  $v \in V'$  it holds  $\deg(v) \geq 3$ . As such,  $|E'| \geq 3\ell$ . Using this the number of critical vertices in  $F$  is bounded via  $3\ell \leq 2|V'| - 4 = 2k + 2\ell - 4$  and thus  $\ell \leq 2k - 4$ . (Note that the addition of any edges between big vertices in  $F$  can only tighten this bound.) As the number of critical vertices in  $F$  is equal to the number of critical clouds in  $\mathcal{P}$  we have shown that there are at most  $2k - 4$  critical clouds in  $\mathcal{P}$ .  $\square$

The next corollary follows together with [Observation 2](#).

**Corollary 1.** *Let  $\mathcal{P}$  be a cloud partition created for a planar graph  $G$ . Then  $\mathcal{P}$  contains  $O(n/\log n)$  critical clouds.*

Note that for leaf or bridge clouds no such bound exists as a cloud partition can contain  $O(n)$  leaf and bridge clouds. Extreme examples include a cloud partition  $\mathcal{P}$  with one big cloud  $C$  and  $n - |C|$  leaf clouds, all adjacent to  $C$  and containing only one vertex. For bridge clouds a similar example partition  $\mathcal{P}$  can be constructed, with two big clouds  $C_1, C_2 \in \mathcal{P}$  and  $n - (|C_1| + |C_2|)$  bridge clouds, all adjacent to both  $C_1$  and  $C_2$ .

**Structure-maintaining minor.** Next we focus on the construction of a specific weighted minor  $F$  of  $G$  with weights  $w(v)$  assigned to each node  $v \in V(F)$ . Intuitively,  $F$  represents a minor of  $G$  that is constructed by repeatedly contracting the vertices in one or more clouds, and the weights keep track of the number of vertices that have been contracted. We call such an  $F$  a *structure-maintaining minor* of  $G$ , with the formal definition outlined shortly. As each such minor is constructed specifically for a cloud partition  $\mathcal{P}$  we say that  $F$  is *induced by  $\mathcal{P}$* . We now define the properties of such a minor and follow with a sketch of a construction. Let  $F$  be a structure-maintaining minor induced by a cloud partition  $\mathcal{P}$ . Denote by  $C_u$  the set of clouds contracted to a node  $u \in V(F)$ . Each node  $u \in V(F)$  is assigned a weight  $w(u) = \sum_{C \in C_u} |C|$ . For each  $u$  one of the following two properties holds: (1)  $|C_u| = 1$ ,  $G[C]$  is connected for  $C \in C_u$  and  $w(u) = \lceil c \log n \rceil + 1$  for some fixed constant  $c$  or (2) each  $C \in C_u$  is adjacent to exactly the same clouds,  $G[\bigcup_{C \in C_u} C]$  contains  $|C_u|$  connected components. Additionally,  $\{v, w\} \in E(F)$  exactly if there exist adjacent clouds  $C_v$  and  $C_w$  with  $C_v \in C_v$  and  $C_w \in C_w$ .

We now outline the construction of  $F$ . Initially,  $F = (V' = \emptyset, E' = \emptyset)$ . For each cloud  $C$  that is big or critical, add a node  $v$  to  $V'$  with  $w(v) = |C|$ . Add edges between  $u, v \in V'$  to  $E'$  exactly if the clouds  $u$  and  $v$  represent are adjacent. We call  $v$  a *big node* if it was added to  $V'$  for a big cloud and *critical node* if it was added for a critical cloud. For each pair of big clouds  $C_1, C_2 \in \mathcal{P}$  connected via one or more bridge clouds, let  $B_{C_1, C_2} \subset \mathcal{P}$  be the set of all such bridge clouds. Add a single node  $v$  to  $V'$  adjacent to the nodes  $u, w$  added to  $V'$  for  $C_1, C_2$  and set  $w(v) = \sum_{B \in B_{C_1, C_2}} |B|$ . We refer to such a node  $v$  as a *meta-bridge node*. For each big cloud  $C \in \mathcal{P}$  adjacent to one or more leaf clouds, denote by  $\mathcal{L}_C \subset \mathcal{P}$  the set of all leaf clouds adjacent to  $C$ . For each such  $C$  add a single node  $v$  to  $V'$  adjacent to the node  $u$  added to  $V'$  for  $C$  with  $w(v) = \sum_{L \in \mathcal{L}_C} |L|$ . We call such a node  $v$  a *meta-leaf node*.

Since we have only  $O(n/\log n)$  big nodes, we also have only  $O(n/\log n)$  critical, meta-bridge and meta-leaf nodes, expressed explicitly in the following lemma. Note that the weight of a meta-bridge or meta-leaf node can be bounded only by  $n$ . We next bound the number of nodes and edges of  $F$ .

**Lemma 5.** *Let  $\mathcal{P}$  be a cloud partition constructed for a planar graph  $G$  and  $F = (V', E')$  the structure-maintaining minor induced by  $\mathcal{P}$ . Then  $|V'| = O(n/\log n)$  and  $O(|E'|) = O(|V'|)$ .*

**Proof.** As  $F$  is a minor of  $G$  it is planar as well and so it holds that  $O(|E'|) = O(|V'|)$ . From [Observation 2](#) we know that the number of big nodes in  $F$  is  $k = O(n/\log n)$ . From the construction of meta-leaf nodes it directly follows that each meta-leaf node is adjacent to exactly one big node and vice-versa. Therefore there are at most  $k$  meta-leaf nodes in  $V'$ . For meta-bridge nodes the proof follows analogous, consider two big nodes  $u, v$  that are adjacent to the same meta-bridge node  $w$ . Then there is no meta-bridge node  $w' \neq w$  adjacent to both  $u$  and  $v$ , as per the construction. By the fact that  $F$  is planar it follows that there are at most  $O(k) = O(n/\log n)$  meta-bridge nodes in  $V'$ . Putting it all together we arrive at  $|V'| = O(n/\log n)$ .  $\square$

**Cloud partitioning structure.** In the following we describe a data structure for a cloud partition and show how it can be constructed and stored in  $O(n)$  time and  $O(n)$  bits. We refer to this data structure as *CP-structure*. A CP-structure constructed for a planar connected graph  $G$  admits the following operations:

- **type( $v$ ):** Given a vertex  $v$  outputs the type of cloud  $v$  belongs to (big, small, critical, bridge or leaf) in  $O(1)$  time.
- **cloud( $v$ ):** Given a vertex  $v$  returns all vertices of the cloud  $C$  in which  $v$  is contained in  $O(|C|)$  time and  $O(|C| \log n)$  space.
- **border( $v, k$ ):** Outputs if the  $k$ th arc out of  $v$ 's adjacency array is part of a border edge in  $O(1)$  time, with  $v \in V(G)$ .

Additionally, the structure allows access to the subgraph  $G'$  of  $G$  induced by all non-border edges of  $\mathcal{P}$ . The graph  $G'$  admits adjacency array access to its vertices and edges. The following lemma describes the runtime and space usage of constructing a cloud partition and a CP-structure.

**Lemma 6.** *Let  $G$  be a planar connected graph. We can compute a cloud partition  $\mathcal{P}$  of  $G$  and a CP-structure of  $\mathcal{P}$  in  $O(n)$  time and  $O(n)$  bits.*

**Proof.** In the following we construct a variety of data structures that are combined to a final CP-structure. For each type of cloud, construct a bitvector of the same name of length  $n$ , i.e., big, small, critical, bridge or leaf. During the following procedure a bit at index  $i$  will be set to 1 in the respective bitvector to indicate that the vertex  $i \in V(G)$  is of the respective type, with all bits set to 0 initially. To track border edges of  $\mathcal{P}$  we maintain a dynamic subgraph  $G'$  of  $G$ , initially containing all edges of  $G$  as outlined in [Section 3](#). A final bitvector **start** of length  $n$  is used during the construction, containing a 1 at index  $i$  exactly if  $i$  was the first vertex discovered in a cloud during the initial construction of the cloud partition  $\mathcal{P}$ .

The first step of the construction is computing  $\mathcal{P}$  by finding all clouds. Starting at an arbitrary unvisited vertex  $v$  set **start**[ $v$ ] = 1 and run a BFS until  $\lceil c \log n \rceil + 1$  unvisited vertices are visited, or no unvisited vertices can be reached. Denote by  $A$  the set of all vertices found during such a BFS. If  $|A| = \lceil c \log n \rceil + 1$  we have found a big cloud. In this case, iterate over the neighborhood of

each vertex  $u \in A$  and mark all edges that are incident to a vertex  $w \notin A$  as border edge by removing it in  $G'$ . Additionally, for all vertices  $u \in A$  set  $\text{big}[u] = 1$ . In the case that  $|A| < \lceil c \log n \rceil + 1$  we have found a small cloud for which we set  $\text{small}[u] = 1$ . In both cases we start the same procedure again from an unvisited vertex  $v$ . If no such vertices remains, we have successfully found  $\mathcal{P}$  and have constructed the bitvectors that allows us to differentiate between small and big clouds. Additionally, the graph  $G'$  is exactly the subgraph of  $G$  induced by non-border edges.

The next step is to construct the bitvectors for critical, bridge and leaf clouds. For each vertex  $v$  that is a start vertex of a big cloud  $C$ , identifiable as  $\text{big}[v] = 1$  and  $\text{start}[v] = 1$ , run a BFS in  $G$ . During this BFS explore all small clouds  $S$  adjacent to  $C$ . If the vertices  $w \in S$  are marked as part of a critical cloud during previous iterations of this step, identifiable via  $\text{critical}[w] = 1$ , ignore  $S$  completely. I.e., at no point traverse edges that are already identified as leading to a critical cloud. Otherwise, for each  $w \in S$  set  $\text{leaf}[w] = 1$  during the first exploration of  $S$ ,  $\text{leaf}[w] = 0$  and  $\text{bridge}[w] = 1$  during the second exploration and  $\text{bridge}[w] = 0$  and  $\text{critical}[w] = 1$  during the third. By this each small cloud and each border edge is accessed at most a constant number of time. Note that each big cloud is traversed once. Afterwards the exact type of cloud a vertex belongs to is accessible via the respective bitvectors in constant time.

The operation  $\text{type}(v)$  is supported in constant time by checking if the bit at index  $v$  is set to 1 in the respective bitvectors constructed for the cloud types in  $O(1)$  time. The operation  $\text{cloud}(v)$  can be accessed by running a BFS from  $v$  in  $G'$  until no vertex can be reached. As  $G'$  contains no border edges it contains each cloud as a connected component. The runtime is linear in the number of vertices of the cloud  $C$  the vertex  $v$  is contained in. The space requirement in terms of bits of a BFS run in a cloud  $C$  is  $O(|C| \log n) = O((\log n)^2)$  as this equals the maximum size of the queue of the BFS. The operation  $\text{border}(v, k)$  can be accessed by simply checking if the edge is present or not in  $G'$ .  $\square$

**Constructing a structure-maintaining minor.** Next we show how to construct a structure-maintaining minor  $F$  induced by a cloud partition  $\mathcal{P}$  of a graph  $G$  in  $O(n)$  time and  $O(n)$  bits of space. By Lemma 5,  $F$  can be stored in  $O(n)$  bits by adjacency lists. We additionally store a bi-directional mapping from each big/critical cloud  $C$  to the node  $v \in V(F)$  added to  $V(F)$  to represent  $C$ . This mapping is stored by a pointer at  $v$  to a single vertex  $v' \in C$  and a pointer at  $v'$  to  $v$ . As there are  $O(n/\log n)$  pointers to store for this bi-directional mapping we use standard pointers of size  $\Theta(\log n)$ . To store the pointers from the direction of a vertex  $v \in V(G)$  we use static-space allocation. Note that the choice of the vertex to store these pointers (per cloud) is arbitrary, but should be fixed. We choose the vertex with the lowest label. For meta-bridge and meta-leaf nodes  $v \in V(F)$  we store a pointer from  $v$  to the lowest labeled vertex  $v'$  amongst all clouds represented by the meta-bridge or meta-leaf node. The details are described in the proof of the next lemma. Next, we define a special operation on  $F$ .

- **expand( $v$ ):** Given a node  $v \in V(F)$  with weight  $w(v)$ , first determine the cloud  $C$  mapped to  $v$  and then return iteratively all vertices part of the clouds  $C \in C$ .

Later we make use of the **expand** operation when translating solutions for problems such as finding separators from  $F$  to a solution to  $G$ . The key difficulty is implementing the **expand** operation for a meta bridge  $v$ . As the clouds represented by  $v$  may induce up to  $n - O(\log n)$  connected components in  $G$  and we store only a pointer to the lowest labeled vertex  $u \in V(G)$  amongst all such clouds represented by  $v$ , we are unable to simply output each cloud without additional information or we must traverse (in the worst case) the entire graph  $G$ . The idea is to store a spanning tree that spans all vertices in clouds represented by  $v$ . As mentioned, since these clouds each induces a connected component, a spanning tree with these clouds does not exist. Therefore, we create a spanning tree by additionally using the vertices of one adjacent big cloud. Traversing all vertices represented by  $v$  can now be done by traversing the respective spanning tree. The key observation for this is that we can direct the edges of  $F$  in such a way that each vertex has up to  $c = O(1)$  outgoing edges (Lemma 3). By this we are able to store all spanning trees in  $O(n)$  bits due to the fact each big cloud is used only by  $c$  spanning trees. Concerning the problem of having  $c$  spanning trees that use the vertices of one big cloud, i.e., that overlap, our approach is to assign each spanning tree a color from a set of  $c$  colors. These spanning trees are then stored in  $c$  dynamic subgraphs, with each subgraph storing all spanning trees of one color.

**Lemma 7.** *Let  $G$  be a connected planar graph and  $\mathcal{P}$  a cloud partition of  $G$  given as a CP-structure. We can construct the structure-maintaining minor  $F$  induced by  $\mathcal{P}$  in  $O(n)$  time and bits such that the **expand** operation on  $v \in V(F)$  runs in  $O(w(v) + \log n)$  time.*

**Proof.** We construct  $F$  iteratively. Initially, we add all big and critical nodes  $v$  and the bi-directional mapping from  $v$  to one vertex  $v'$  in the cloud  $v$  represents. During this process, we maintain a bitvector to mark vertices of  $G$  as visited. An additional bitvector  $X$  is maintained for marking the lowest numbered vertex in each big or critical cloud. Starting at the lowest labeled vertex  $v$ , obtain the cloud  $C$  with  $v \in C$  via the  $C = \mathcal{P}.\text{cloud}(v)$  operation. Mark all vertices of  $C$  as visited, add a node  $v'$  to  $V(F)$  and store a pointer at  $v'$  to  $v$  and store a pointer from  $v$  to  $v'$  in a standard linked list  $L$  and set  $X[v] = 1$ . Simply iterate this process until all big and critical nodes are added to  $V(F)$ . Note that by this construction the lowest labeled vertex of a cloud is always the vertex for which the bi-directional mapping is constructed, as we iterate over the bitvector for visited vertices in order. Constructing an indexable dictionary for  $X$  and transforming  $L$  into an array (in  $O(|L|)$  time and  $O(|L| \log n)$  bits) gives us exactly the static space allocation for pointers from vertices in  $V(G)$  to nodes in  $V(F)$ .

Now we add all edges between big and critical nodes. We maintain two bitvectors **complete** and **discovered** for vertices, initially containing only 0 bits. In **complete** we mark processed big clouds and in **discovered** we mark clouds discovered from a big cloud during the iteration to ensure that these clouds are only visited once. Iteratively, start at an arbitrary vertex  $v$  that is not marked **complete** and is contained in a big or critical cloud. Obtain the cloud  $C$  via  $\mathcal{P}.\text{cloud}(v)$ . Mark all vertices of  $C$  as **complete**. For each border edge  $\{u, w\}$  with  $u \in C$  and  $w$  in a big or critical cloud  $C'$  adjacent to  $C$  and for which  $w$  is not marked as **discovered** obtain

$C'$  via  $\mathcal{P}.\text{cloud}(w)$ . Mark all vertices in  $C'$  as discovered and obtain the node  $w' \in V(F)$  that represents  $C'$  via the bi-directional mapping, stored for the lowest labeled vertex of  $C'$ . Add the arc  $v'w'$  to the adjacency list of  $v'$ . Note that the arc  $w'v'$  is added during a different iteration, i.e., when we start at  $C'$  and find  $C$  as an adjacent cloud. Once this process has been completed for all border edges incident to  $C$  mark all adjacent big or critical clouds as no longer discovered by an analogous process. The entire process is iterated until no vertices  $v$  in critical or big clouds remain that are not marked complete. The weights for each node  $v$  added to  $V(F)$  during this step can be determined and stored during the run by simply counting the vertices in the cloud that  $v$  represents. We so have added  $O(n/\log n)$  nodes and edges to  $F$  during this process. For each edge  $\{u, v\} \in E(F)$  (as the two arcs  $uv$  and  $vu$ ) we have explored the clouds of each endpoint a constant number of times via the  $\text{cloud}$  operation, which runs in linear time. As each such cloud has size  $O(\log n)$  and  $|E(F)| = O(n/\log n)$  the overall runtime is  $O(n)$ .

In the next step we add all meta-leaf nodes to  $F$ . As in the previous step we maintain the same bitvectors and re-initialize them. Iteratively, start at an arbitrary vertex  $v$  that is not marked complete and is contained in a big cloud  $C$ , obtainable via  $\mathcal{P}.\text{cloud}(v)$ . For each border edge  $\{u, w\}$  with  $u \in C$  that is incident to a leaf cloud and for which  $w$  is not marked as discovered obtain the cloud  $C'$  adjacent to  $C$  via  $\mathcal{P}.\text{cloud}(w)$  and mark all vertices of  $C'$  as discovered. Once all border edges have been iterated this way, add a node  $u$  to  $V(F)$  incident to  $v'$ . The weight of  $u$  is set to the number of vertices in all leaf clouds adjacent to  $C$ . Additionally, we store a pointer from  $u$  to the lowest labeled vertex  $u'$  amongst all leaf clouds which are represented by  $u$ . Repeat the entire process until no (unvisited vertices in) big clouds remain. As in the previous step, for each edge  $\{u, v\}$  added to  $E(F)$  we iterate over the clouds of  $u$  and  $v$  a constant number of time and therefore have a runtime of  $O(n)$ .

Finally, we need to add all meta-bridge nodes to  $F$ . Similar as to the previous steps we start at a big cloud  $C$  and explore all neighboring bridge clouds  $B$  and the big clouds connected to  $C$  via the bridge clouds. Assume the same bitvectors as in the previous steps are available. We work on two dynamic subgraphs  $G_a, G_b$  of  $G$ , with  $G_a$  initially being a copy of  $G$  and  $G_b$  being the empty graph. In  $G_a$  we remove border edges incident to vertices in bridge clouds for which we have already added a meta-bridge node to  $F$  to avoid unnecessary explorations. During the process we start at an arbitrary big cloud  $C$  not marked complete. We then construct  $G_b$  as the graph induced by  $C$ , all bridge clouds  $B$  adjacent to  $C$  and all big clouds connected to  $C$  via  $B$ . In  $G_b$  we start at an arbitrary bridge cloud  $B$  adjacent to  $C$ , remove the edges incident to vertices of  $B$  and vertices of  $C$ , and traverse to the big cloud  $C'$  connected to  $C$  via  $B$ . From  $C'$  we then explore all adjacent bridge clouds  $B$ , removing all traversed edges and all edges adjacent to vertices of  $B$  in  $G_b$  and  $G_a$ . Once finished, we add a meta-bridge node  $v$  to  $F$  and the edges  $\{v, w\}$  and  $\{v, u\}$  with  $w$  being the node added to  $V(F)$  for  $C$  and  $u$  being the node added to  $V(F)$  for  $C'$ . The weight of  $v$  is simply the number of vertices traversed in the bridge clouds represented by  $v$ . Additionally, we store a pointer from  $v$  to the lowest labeled vertex  $v'$  amongst all bridge clouds which are represented by  $v$ . Finally, we mark all vertices of  $C$  as complete and repeat the process from the next big cloud not yet marked complete. For each meta-bridge node  $v$  added to  $V(F)$  we explore the two clouds  $C, C'$  represented by the two big nodes adjacent to  $v$  once. Additionally, we explore all vertices in bridge clouds represented by  $v$  a constant number of time. As we add  $O(n/\log n)$  meta-bridge nodes overall the runtime is  $O(n)$ .

It remains to show how to implement the `expand` operation. For big and critical nodes  $v \in V(F)$  the `expand` operation is trivially solved by obtaining the vertex  $v'$  mapped to  $v$  and calling  $\mathcal{P}.\text{cloud}(v')$ . We therefore now focus only on meta-bridge and meta-leaf nodes. We first give a non-technical overview of our technique followed by a technical implementation and analysis. Denote by  $C_u$  the set of clouds represented by a meta-leaf or meta-bridge node  $u \in V(F)$ . Intuitively, our goal is to create a tree  $T_u$  spanning over all vertices in  $\cup_{C \in C_u} C$ . As the clouds in  $C_u$  induce disconnected components in  $G$ , we must add vertices of one cloud  $C_0$  to make  $T_u$  a tree where  $C_0 \in \mathcal{P}$  is one of the clouds adjacent to all  $C \in C_u$  in  $F$ . Once such a spanning tree is available for all meta-bridge and meta-leaf nodes  $u$ , the `expand` operation can simply be achieved by traversing  $T_u$  via a depth-first search. It is easy to see that the height of each  $T_u$  is  $O(\log n)$  since  $F[C_0 \cup C_u]$  is a star so that we can easily execute a standard DFS and only need  $O(n)$  bits.

For meta-leaf nodes storing these spanning trees can easily be achieved in a single dynamic subgraph  $G_{\text{leaf}}$  as all  $T_u$  are pairwise disjoint.

For meta-bridge nodes we make use of the fact that any separable graph can be directed in such a way that each vertex has at most in-degree  $d$  for some graph class dependent constant  $d$  as per Lemma 3. We first create a minor  $F'$  of  $F$  that contains only big nodes and an edge between two big nodes  $u, v$  exactly if there is a meta-bridge node adjacent to both  $u$  and  $v$ . Finally, we split each edge by adding the corresponding meta-bridge node in the middle of the edge and direct the two split edges in the direction of the original edge.

Recall that each  $T_u$  constructed for the bridge clouds  $C_u$  represented by some meta-bridge node  $u$  spans only one big cloud  $C_v$ . We choose  $C_v$  as the cloud such that there is a directed edge  $uv$  in  $F'$  between the nodes  $u$  and  $v$  corresponding to  $C_u$  and  $C_v$ , respectively. By this there are at most  $d$  meta-bridge nodes that use the same big cloud. Observe that all spanning trees that do not use the same big cloud are disjoint. From this, we are able to store all spanning trees in  $d$  dynamic subgraphs  $G_1, \dots, G_d$ . In intuitive terms, we can color all spanning trees with  $d$  colors such that any two spanning trees  $T, T'$  are disjoint if they are assigned the same color. The spanning trees can be constructed in intermediate computations while repeating the same process for adding the meta-bridge nodes  $v$  to  $V(F)$  and their incident edges. Analogously (and simpler) the same is true for meta-leaf nodes. For each meta-bridge node  $v$  we store a pointer to the one dynamic subgraph  $G_i \in G_1, \dots, G_d$  in which  $T_u$  is contained. To traverse  $T_u$  we then obtain the vertex  $v' \in V(G)$  mapped to  $v$  and start a depth-first search from  $v$  in  $G_i$ . For each  $T_u$  it holds  $|V(T_u)| = w(u) + O(\log n)$ . Constructing each  $T_u$  can be done in  $O(w(u) + \log n)$  time, and there are  $O(n/\log n)$  spanning trees overall, as each node in  $F$  results in up to  $d = O(1)$  spanning trees and there are  $O(n/\log n)$  nodes in  $F$ . As such, it takes  $O(n)$  time and uses  $O(n)$  bits.  $\square$

We refer to the combined data structure of Lemma 6 and Lemma 7 as a *cloud decomposition*  $(F, \mathcal{P})$  of  $G$ .

## 5. Applications: succinctly encoded planar graphs

In this section we show how a cloud decomposition  $(F, \mathcal{P})$  constructed for a planar graph  $G$  can be used to find a balanced separator of size  $O(\sqrt{n \log n})$  in  $O(n/\log n)$  time and  $O(n)$  bits, which in turn can be used to construct a succinct encoding of planar graphs, described later. Afterwards we show how such a cloud decomposition can be used together with the search for balanced separators to compute a tree decomposition with width  $O(n^{1/2+\epsilon})$  for any  $\epsilon > 0$  of a planar graph  $G$ . We make use of an  $O(n)$ -time  $O(n \log n)$ -bits algorithm for finding 2/3-balanced separators of size  $O(\sqrt{n})$  in weighted planar graphs as long as no vertex has a weight more than  $1/3$  times the total weight [8,51,52]. The next theorem shows how a balanced separator can be constructed for a planar graph with the help of a cloud decomposition. The idea is to run a slightly modified standard algorithm for finding balanced separators  $S$  on the structure-maintaining minor  $F$  and translating  $S$  to a separator  $S'$  of  $G$ .

**Theorem 1.** *Let  $G = (V, E)$  be a planar graph and  $(F = (V', E'), \mathcal{P})$  a cloud decomposition of  $G$ . We can construct a balanced separator  $S$  of  $G$  with size  $O(\sqrt{n \log n})$  in  $O(n/\log n)$  time using  $O(n)$  bits.*

**Proof.** Start to search for a separator  $S \subset V'$  on the weighted graph  $F$  in time  $O(|V'|) = O(n/\log n)$  and  $O(|V'| \log n) = O(n)$  bits. Denote by  $S \subset \mathcal{P}$  the clouds of  $\mathcal{P}$  represented by nodes of  $S$  and by  $w(S)$  the sum of the weights  $w(v)$  of all  $v \in S$ . We can construct a separator  $S'$  for  $G$  via  $S' = \bigcup_{C \in S} C$ . We store  $S'$  in a bitvector of length  $n$ , initialized to all 0-bits in  $O(n/\log n)$  time. For each  $v \in S$  we obtain an iterator over the clouds  $v$  represents via  $F.\text{expand}(v)$  and simply add the vertices returned by the iterator to  $S'$  in  $O(w(s))$  time. Note that  $|S'| = O(|S| \log n) = O(\sqrt{n/\log n} \log n) = O(\sqrt{n \log n})$ .

A balanced separator  $S$  of  $F$  can only be found if no node  $v \in V'$  has weight larger than  $1/3$  times the total weight. Only meta-leaf or meta-bridge nodes can have such large weights. If such a node exists, we split it into up to three nodes by distributing its clouds as equally as possible to the three nodes and run the search on the modified graph.  $\square$

Due to Blelloch and Farzan [13] there exists a succinct encoding of an arbitrary separable graph  $G = (V, E)$  that provides constant-time adjacency, degree and neighborhood queries. The encoding is constructed via recursive application of a separator algorithm such that  $V$  is split into two sets  $A$  and  $B$  via a separator  $S$ . This recursion is continued for  $G_a = G[A \cup S]$  and  $G_b = G[B \cup S]$  until the remaining graphs are of size at most  $\log^\delta n$  for a graph class specific constant  $\delta$ , which they refer to as *mini graphs*. In several papers, what is constructed here is referred to as a *separator hierarchy*. For technical reasons the edges between vertices of  $S$  are only included in  $G_a$  and not  $G_b$ . The graph  $G$  is then encoded by a combination of these mini graphs, which in turn are further decomposed by the same recursive separator search into *micro graphs*, which are then small enough to be handled via lookup tables encoding every possible micro graph and in turn are used to encode the mini graphs. Constructing these mini and micro graphs can be done in  $O(n)$  time for a planar graph  $G$  using the techniques described by Blelloch and Farzan combined with the algorithm of Goodrich for constructing a separator hierarchy [23], but we want to use only  $O(n)$  bits and maintain the runtime. The main result of [13] is summed up by the next theorem.

**Theorem 2 ([13]).** *Any family of separable graphs with entropy  $\mathcal{H}(n)$  can be succinctly encoded in  $\mathcal{H}(n) + o(n)$  bits such that adjacency, neighborhood, and degree queries are supported in constant time.*

One important point of the construction is the fact that the number of *duplicate vertices* in each mini graph is bounded. A duplicate is a vertex that is contained in more than one mini graph. A vertex becomes a duplicate any time it is part of a separator  $S$ , as it is then contained in both  $G_a$  and  $G_b$ . The exact bound is defined by the next lemma.

**Lemma 8 ([13]).** *The number of mini graphs is  $\Theta(n/\log^\delta n)$ . The total number of duplicates among mini graphs is  $O(n/\log^2 n)$ . The sum of number of vertices of mini graph together is  $n + O(n/\log^2 n)$ .*

Our goal is to execute the recursive decomposition of the graph  $G$  via a cloud decomposition  $(F, \mathcal{P})$  by recursively searching for separators in  $F$  until the entire weight of the remaining graphs is less than  $\log^\delta n$ . These small subgraphs of  $F$  then are expanded to be exactly the mini graphs of  $G$  we aim to find. For each mini graph a new cloud decomposition is constructed, and the recursive separator search is repeated for each mini graph to construct the micro graphs.

**Lemma 9.** *Let  $G$  be a planar graph and  $(F, \mathcal{P})$  a cloud decomposition of  $G$ . We can output all mini graphs of  $G$  in  $O(n)$  time.*

**Proof.** The idea is to search for a balanced separator  $S$  of  $F$  that partitions  $V(F)$  into three parts  $\{A, S, B\}$ . Continue recursively for  $F' = F[A \cup S]$  and  $F'' = F[B \cup S]$  (removing edges between nodes of  $S$  from one of these instances). Stop the recursion once  $F'$  has an overall weight  $< \log^\delta n$ . The nodes  $v \in V(F')$  are then expanded to their respective clouds  $C \in \mathcal{P}$ . The vertex induced subgraph of  $G$  over the vertices in all these clouds  $C$  is then a mini graph that we output. In the following, let  $F'$  be the input graph of the current recursive call of our procedure. We denote by  $w(v)$  the weight of a node and by  $w(F')$  the total weight of all nodes of  $V(F')$ . Assume for now that  $V(F')$  contains no node of weight  $w(v) > 1/3 w(F')$ . In this case a balanced separator  $S$  of  $F'$  of weight  $O(\sqrt{w(F')})$  can easily be found in  $O(|V(F')|)$  time [8,51,52], and we can progress recursively as outlined. If we do not encounter a problem with nodes of too large weight at any point, the entire recursive algorithm takes  $O(|V(F)| \log n) = O(n)$  time.

Analogous to the proof of Theorem 1 we need to consider the special case of a leaf and bridge node  $v$  exceeding the weight limit of  $\beta w(F')$  (other nodes can not exceed this limit). In this case the neighborhood  $S$  of  $v$  in  $F'$  is a separator of  $V(F')$  into  $\{A, S, B\}$  with  $A = \{v\}$  and  $B = V(G) \setminus (A \cup S)$ . Denote by  $S$  the set of all clouds represented by nodes of  $S$  and by  $C_v$  the set of all clouds represented by  $v$ . Note that  $S$  contains one cloud if  $v$  is a meta-leaf node, and two clouds if  $v$  is a meta-bridge node. We can now output mini graphs  $G_m$  directly, with each  $G_m$  being the vertex induced subgraph over  $S$  and some  $C \in C_v$ . We collect the vertices

of each  $G_m$  as follows. Start with a set  $M$  initialized to  $M = \bigcup_{S \in \mathcal{S}} S$ , i.e., the separator. Now iterate over  $C \in \mathcal{C}_v$  and set  $M = M \cup C$  if  $|M \cup C| < \log^\delta n$ . If the threshold is reached or the iteration is over, we have found a mini graph  $G_m = G[M]$ . In this case reset  $M = \bigcup_{S \in \mathcal{S}} S$  and continue. This entire process takes  $O(|S| + |\mathcal{C}_v|) = O(\log n + w(v)) = O(w(F'))$  time. The recursion now only has to continue for  $F'[V(F') - v]$ , which contains less than  $2/3$  times the total weight of  $F'$ .

In any case the recursion is stopped once the input graph has weight  $< \log^\delta n$  at which point we have found a mini graph, which can be output by translating nodes of the input graph  $F'$  to the respective clouds of  $G$ , which induce the mini graph  $G_m$  that we want to output. Note that obtaining the vertices  $M$  of a mini graph  $G_m$  can easily be achieved in  $O(|V(G_m)|)$  time by translating the nodes of  $F'$  to their respective clouds in  $O(\log^\delta n)$  time. For obtaining the edges between vertices of  $M$  we need to create a graph  $G_d$  which is a directed version of  $G$  such that each vertex has bounded in-degree (Lemma 3). Using  $G_d$  we are able to translate the vertices for each mini graph linear in the size of the mini graph. Note that the graph  $G_d$  needs to be created only once as a global structure in  $O(n)$  time and  $O(n)$  bits by using a dynamic subgraph to store  $G_d$ . The overall runtime of all recursive calls is then  $O(|V(F)| \log n) + O(n) = O((n/\log n) \log n) = O(n)$ .  $\square$

The bound on the number of duplicates due to Lemma 8 can be upheld with some care.

**Lemma 10.** *Let  $G$  be a planar graph and  $(F, \mathcal{P})$  a cloud decomposition of  $G$ . The total number of duplicate vertices among the mini graphs of the cloud decomposition is  $O(n/\log n)$ .*

**Proof.** As mentioned by Blelloch et al. [13] it suffices if the separator that is found is a polylogarithmic approximation when encoding an arbitrary separable graph, i.e., if the graph that is to be encoded admits to an  $O(n^c)$ -separator theorem for some  $c < 1$ , it suffices to find a separator of size  $O(n^c \log^k n) = O(n^{c'})$  for some  $k > 1$  and some  $c' > c$ . When using the standard method of finding recursive separators, the size of each separator is dependent on the input graph of each recursive call to the separator algorithm. As an example consider an input graph  $G'$  of size  $\log^\delta n$ , which is the largest graph that can occur as an input during the recursion (any smaller graph is a mini graph). When searching for a separator for  $G'$  a separator of size  $O((\log^\delta n)^c)$  is found for some  $c < 1$ . Consider our space-efficient separator search that operates on a cloud decomposition  $(F, \mathcal{P})$  constructed for  $G$ , with a recursive separator search on  $F$  instead of  $G$ . In this case, the largest graph is defined by the total weight of the input graph  $F'$  of the recursion. If the total weight is  $\log^\delta n$ , then a separator  $S$  of  $F'$  of size  $O((\log^{\delta-1} n)^c)$  with total weight  $O((\log^{\delta-1} n)^c \log n)$  is found in  $F'$ . The key observation is that the  $\log n$  factor at the end is independent of the size of the input graph  $F'$ . As outlined, the vertices that are contained in a separator are exactly those that are duplicate vertices in the mini graphs of the encoding. By using our scheme to output the mini graphs increases the number of duplicate vertices by such a large factor that it invalidates Lemma 8. We therefore adjust the constant  $\delta$  specific to the graph class—see below.

Intuitively, the encoding due to Blelloch and Farzan can encode arbitrary separable graphs. For these separable graphs, the  $\delta$  values used can become arbitrarily large to adhere to Lemma 8. Therefore, we are able to increase it. In Lemma 8 Blelloch and Farzan mention a constant  $\delta$ . In their work they set  $\delta = 1/(1 - c)$  when encoding a graph  $G$  that admits to an  $O(n^c)$ -separator theorem. For planar graphs  $c = 1/2$  and therefore  $\delta = 4$ . When using a recursive separator algorithm until mini graphs of size  $< \log^\delta n = \log^4 n$  are found, the number of total mini graphs is  $O(n/\log^4 n)$ , which is equal to the number of leaf nodes of the recursion tree. This tree has depth  $O(\log n)$  and as it is binary,  $O(n/\log^4 n)$  total nodes. The number of duplicate vertices can then be bounded by  $O(n/\log^4 n \cdot (\log^4 n)^{1/2}) = O(n/\log^2 n)$  as per Blelloch and Farzan. When using our space-efficient approach we have to increase the  $\delta$  value to adhere to this bound, as the number of duplicates is now bounded by  $O(n/\log^4 n \cdot (\log^4 n)^{1/2} \log n)$  due to the constant  $O(\log n)$  factor that is independent of the smaller input graphs of the recursive separator search and only dependent on the size of the first input graph  $G$ .

This would mean a total number of duplicate vertices among mini graphs of  $O(n/\log n)$ . To mitigate this, we replace  $\delta$  with  $\delta + 2$  to bound the number of duplicates by  $O(n/\log^6 n \cdot (\log^6 n)^{1/2} \log n) = O(n/\log^2 n)$ . Note that arbitrary large  $\delta$  values can be chosen for the encoding as separable graphs can be encoded with arbitrarily small  $c$  values [13].  $\square$

The full encoding in  $O(n)$  time can be done by using the algorithm of Lemma 9 to output all mini graphs. For every mini graph  $G_m$  we again construct a cloud decomposition in linear time and use the algorithm of Lemma 9 to find the micro graphs of  $G_m$ . All micro graphs are encoded using the table lookup scheme outlined in Blelloch and Farzan [13]. All other structures needed for the encoding can be constructed in  $O(n)$  time and  $O(n)$  bits easily, as they are simple indexable dictionaries over vertices of the micro graphs and mini graphs or small lookup tables which can be initialized in time linear in their size, which is  $O(n)$ . From this description the next theorem follows.

**Theorem 3.** *A planar graph  $G$  with entropy  $H(n)$  can be succinctly encoded in  $H(n) + o(n)$  bits such that adjacency, neighborhood, and degree queries are supported in constant time. The encoding can be constructed in  $O(n)$  time using  $O(n)$  bits.*

## 6. Applications: planar tree decompositions

We next present a simple modification of the recursive separator search of Lemma 9 using standard techniques to output a tree decomposition of  $G$ . Let  $(F, \mathcal{P})$  be a cloud decomposition of  $G$ . Each balanced separator  $S$  of  $F$  induces a bag in a tree decomposition of  $F$  via the following recursive relation. Let  $F'$  be the input graph used in the recursive calls, initially  $F' = F$ . Additionally, maintain a set  $X$  containing vertices contained in separators found in previous recursive calls, initially  $X = \emptyset$ . When a separator  $S$  is found for  $F$  that partitions  $V(F)$  into three sets  $\{A, S, B\}$  output the next bag of the tree decomposition of  $F$  as  $S \cup X$ . Continue the recursion for the input  $F'[S \cup A]$  and  $X = (X \cup S) \cap A$  and the input  $F'[S \cup B]$  and  $X = (X \cup S) \cap B$ . Note that this tree decomposition of  $F$  has width  $O(\sqrt{|V(F)|})$ . This tree decomposition can easily be expanded to a tree decomposition of  $G$  by expanding all vertices of a bag

$B$  to their respective clouds. The width of this expanded tree decomposition is  $O(\sqrt{|V(F)|} \log n) = O(n^{1/2+\epsilon})$  for any  $\epsilon > 0$ . Each bag can be output in  $O(n^{1/2+\epsilon})$  time and there are  $O(n)$  bags. This description allows the next corollary.

**Corollary 2.** *Let  $G$  be a planar graph. We can compute and output the bags of a tree decomposition with width  $O(n^{1/2+\epsilon})$  for any  $\epsilon > 0$  in time linear in the size of the tree decomposition  $O(n^{3/2+\epsilon})$  using  $O(n)$  bits.*

## 7. Generalizing to $H$ -minor-free graphs

To generalize the previous results to  $H$ -minor-free graphs from planar graphs we must generalize cloud partitions and their induced minors. Recall that a cloud partition  $\mathcal{P}$  for a planar graph  $G$  contains  $O(n/\log n)$  critical clouds. Critical clouds are exactly those of size  $< c \log n + 1$  and with  $\geq 3$  adjacent big clouds. We define a  $\phi$ -critical cloud as a small cloud adjacent to  $\geq \phi$  big clouds. The idea of the proof is the same as for the proof of Lemma 4, but we now have a bound  $\phi$  that depends on the graph class. The existence of the bound  $\phi$  is due to the fact that a separable graph  $G = (V, E)$  has bound density  $d$ , i.e.,  $|E| \leq d|V|$  for some fixed constant  $d$ . In particular we show that setting  $\phi$  as  $d + 1$  gives us the desired properties outlined in the following.

**Lemma 11.** *Let  $\mathcal{G}$  be a  $H$ -minor-free family of graphs for some fixed graph  $H$  and let  $d$  be the maximal density of a graph in  $\mathcal{G}$ . There exists a cloud partition  $\mathcal{P}$  for each graph  $G \in \mathcal{G}$  such that  $\mathcal{P}$  contains  $O(n/\log n)$   $\phi$ -critical clouds for  $\phi = d + 1$ .*

**Proof.** Let  $\mathcal{P}$  be a cloud partition of a graph  $G \in \mathcal{G}$  and let  $F = (V', E')$  be the graph constructed by contracting the vertices of each cloud  $C \in \mathcal{P}$  to a single vertex  $v \in V'$ . Two vertices  $u, v \in V'$  are adjacent in  $F$  exactly if  $C_u$  and  $C_v$  are adjacent. We refer to the vertices of  $F$  introduced for  $\phi$ -critical clouds as  $\phi$ -critical vertices and vertices introduced for big clouds as big vertices. Recall that a  $\phi$ -critical cloud is a cloud of size  $< [c \log n] + 1$  for some chosen constant  $c$  with  $\geq \phi$  adjacent big clouds. Remove all vertices that are not big vertices or  $\phi$ -critical vertices and all edges incident to two big vertices from  $F$ . As  $\mathcal{G}$  is minor-closed and  $F$  is a minor of  $G$ ,  $F$  must be part of  $\mathcal{G}$  as well. It follows that  $|E'| \leq d|V'|$  with  $d$  being the maximal density of any graph part of  $\mathcal{G}$ . Denote by  $h$  the number of  $\phi$ -critical vertices and  $k$  the number of big vertices in  $F$ . As the degree of each  $\phi$ -critical vertex is  $\geq \phi$  it holds that  $|E'| \geq \phi h$ . Using this we can bound the number of  $\phi$ -critical vertices in  $F$  via  $\phi h \leq d|V'| = dh + dk$  and thus  $h \leq dk/(\phi - d)$ . For  $\phi = d + 1$  it then follows  $h \leq dk$ . As there are  $O(n/\log n)$  big clouds in  $\mathcal{P}$  it follows that there are  $O(n/\log n)$   $\phi$ -critical clouds in  $\mathcal{P}$ .  $\square$

It remains to show how small clouds adjacent to  $< \phi$  big clouds are to be handled. For planar graphs such clouds are exactly the bridge and leaf clouds. To generalize from planar graphs to  $H$ -minor-free graphs we must in turn generalize bridge clouds similarly to the generalization from critical to  $\phi$ -critical clouds. We call such generalized bridge clouds  $\phi$ -bridge clouds, which we define as small clouds adjacent to  $< \phi$ , but  $> 2$  big clouds. Analogous to regular bridge clouds, there can be  $O(n)$  such clouds in a cloud partition for an  $H$ -minor-free graph. We refer to cloud partitions with the additional labeling of clouds as  $\phi$ -critical and  $\phi$ -bridge as a *generalized cloud partition*  $\mathcal{P}$ .

To construct the structure-maintaining minor  $F$  induced by a generalized cloud partition  $\mathcal{P}$  we can use the same strategy for bridge clouds and leaf clouds. For  $\phi$ -critical clouds the strategy also remains the same as for critical clouds in regular cloud partitions. For  $\phi$ -bridge clouds we introduce a generalized version of the meta-bridge node to  $F$ , which we call  $\phi$ -meta-bridge node. Note that each  $\phi$ -meta-bridge node has degree  $< \phi$  in  $F$ . For adding all  $\phi$ -meta bridge nodes we iterate over  $i \in (3, \dots, \phi - 1)$ . For each  $i$  we add all  $\phi$ -meta bridges with degree  $i$  to  $V(F)$ . Adding all degree- $i$   $\phi$ -meta bridge nodes  $v$  takes  $O(in)$  time as for each neighbor of  $v$  the respective clouds must be explored. Thus, an overall time of  $O(\phi^2 n)$  is needed during the construction. As described above Lemma 7, we store spanning trees for meta-bridge nodes and meta-leaf nodes. We store and construct the exact same spanning trees for the  $\phi$ -meta-bridge nodes. These spanning trees are stored in  $c$  different dynamic subgraphs, with  $c$  being some constant, as described in Lemma 3. Storing these dynamic subgraphs takes  $O(cn) = O(\phi n)$  bits of space.

**Lemma 12.** *Let  $G$  be a  $H$ -minor-free graph for some fixed graph  $H$  with a generalized cloud partition  $\mathcal{P}$ . We can construct the minor induced by  $\mathcal{P}$  in  $O(\phi^2 n)$  time and  $O(\phi n)$  bits with  $\phi$  being a graph class dependent constant.*

Analogous for planar graphs, we call the combination of a generalized cloud partition  $\mathcal{P}$  and a minor induced by  $\mathcal{P}$  a  $\phi$ -cloud decomposition. This allows us to generalize Theorems 1, 2 and Corollary 2 to  $H$ -minor-free graphs using such generalized cloud partitions. For generalizing the proofs of these theorems and the corollary we only must handle the new case of  $\phi$ -meta-bridge nodes exceeding the weight threshold during the separator search, as all other cases work analogous. Recall that we must handle the case when a node  $v \in V(F)$  has too large of a weight, and thus no balanced separator can be found. A new additional case now arises when a  $\phi$ -meta-bridge node  $v$  exceeds the weight threshold which is handled exactly the same as meta-bridge and meta-leaf nodes. In detail, the neighborhood  $S$  of  $v$  in  $F$  is a separator that separates  $v$  from  $V(F) \setminus S$ . Expanding  $S$  to a separator  $S'$  of  $G$  then separates all vertices in clouds represented by  $v$  from the rest of the graph. The balanced separator  $S'$  then contains  $O(\phi \log n)$  vertices. For the following theorems we make use of the linear time  $O(n^{2/3})$ -separator theorem for  $H$ -minor-free graphs [11].

**Theorem 4.** *Let  $G$  be a  $H$ -minor-free graph for some fixed graph  $H$ , let  $\phi$  be a graph class dependent constant and  $(F, \mathcal{P})$  a generalized cloud partition of  $G$ . We can compute a balanced separator  $S$  of  $G$  with size  $O(n^{2/3+\epsilon})$  for any  $\epsilon > 0$  in  $O(n/\log n)$  time using  $O(n)$  bits.*

**Theorem 5.** *A  $H$ -minor-free graph for some fixed graph  $H$  with entropy  $\mathcal{H}(n)$  can be succinctly encoded in  $\mathcal{H}(n) + o(n)$  bits such that adjacency, neighborhood, and degree queries are supported in constant time. The construction of the encoding of takes  $O(\phi^2 n)$  time and uses  $O(\phi n)$  bits with  $\phi$  a graph class dependent constant.*

**Corollary 3.** *Let  $G$  be a  $H$ -minor-free graph for some fixed graph  $H$ ,  $\phi$  a graph class dependent constant and  $(F, \mathcal{P})$  a generalized cloud partition of  $G$ . We can compute a tree decomposition of  $G$  with width  $O(n^{2/3+\epsilon})$  for any  $\epsilon > 0$  in  $O(n^{5/3+\epsilon})$  time using  $O(n)$  bits.*

## 8. Conclusion

In this work, we presented a space-efficient decomposition technique called cloud decomposition for planar and  $H$ -minor-free graphs. Our technique is based on a simple algorithm that iteratively performs breadth-first searches. Using this partition, we construct a minor of the input graph that preserves the main structure of the original input graph. In total, we partition the vertices into connected subgraphs of size  $O(\log n)$  and construct a structure-maintaining minor  $F$  of size  $O(n/\log n)$ . This decomposition can then be used to execute standard algorithms, such as computing balanced separators or constructing tree decompositions, with minimal modifications. Finally, we demonstrated that the succinct encoding scheme proposed by Blelloch and Farzan [13] can be implemented in time and space complexity of  $O(n)$ .

Additionally, we extended our framework to  $H$ -minor-free graphs, enabling broader applications. This includes a space-efficient construction of the succinct encoding scheme by Blelloch and Farzan [13], reducing runtime and space by a factor of  $\Omega(\log n)$ . Possible research directions include applying our coarsening scheme to problems outside of computing separators. While we do not rely on embeddings, it would be interesting to research whether the availability of a planar embedding would enable us to implement space-efficient variants of more advanced algorithms. For example, the decomposition algorithm of Goodrich [23] computes a recursive separator hierarchy in linear time using  $O(n \log n)$  bits. Such a result would eliminate the additional factor of  $O(\log n)$  in the size of the balanced separators that can be constructed using our cloud decomposition scheme.

## CRedit authorship contribution statement

**Nina Hammer:** Writing – review & editing, Writing – original draft; **Frank Kammer:** Writing – review & editing, Writing – original draft; **Johannes Meintrup:** Writing – review & editing, Writing – original draft.

## Data availability

No data was used for the research described in the article.

## Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Johannes Meintrup reports financial support was provided by DFG - German Research Foundation. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## References

- [1] B. Strasser, D. Wagner, T. Zeitz, Space-efficient, fast and exact routing in time-dependent road networks, in: Proceedings of the 28th Annual European Symposium on Algorithms (ESA 2020), 173 of *Leibniz International Proceedings in Informatics (LIPIcs)*, Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2020, pp. 81:1–81:14. <https://doi.org/10.4230/LIPIcs.ESA.2020.81>
- [2] V. Floreskul, K. Tretyakov, M. Dumas, Memory-efficient fast shortest path estimation in large social networks, Proc. Int. AAAI Conf. Web Soc. Media (PMLR 2020) 8 (2014) 91–100. <https://doi.org/10.1609/icwsm.v8i1.14532>
- [3] A. Chhabra, M. Fonseca Faraj, C. Schulz, D. Seemaier, Buffered streaming edge partitioning, in: Proceedings of the 22nd International Symposium on Experimental Algorithms (SEA 2024), 301 of *Leibniz International Proceedings in Informatics (LIPIcs)*, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, pp. 5:1–5:21. <https://doi.org/10.4230/LIPIcs.SEA.2024.5>
- [4] T. Hagerup, Space-efficient DFS and applications to connectivity problems: simpler, leaner, faster, *Algorithmica* 82 (4) (2020) 1033–1056. <https://doi.org/10.1007/s00453-019-00629-x>
- [5] T. Asano, T. Izumi, M. Kiyomi, M. Konagaya, H. Ono, Y. Otachi, P. Schweitzer, J. Tarui, R. Uehara, Depth-first search using  $O(n)$  bits, in: *Algorithms and Computation*, Springer, 2014, pp. 553–564. [https://doi.org/10.1007/978-3-319-13075-0\\_44](https://doi.org/10.1007/978-3-319-13075-0_44)
- [6] A. Elmasry, T. Hagerup, F. Kammer, Space-efficient basic graph algorithms, in: E.W. Mayr, N. Ollinger (Eds.), Proceedings of the 32nd International Symposium on Theoretical Aspects of Computer Science (STACS 2015), 30 of *Leibniz International Proceedings in Informatics (LIPIcs)*, Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2015, pp. 288–301. <http://drops.dagstuhl.de/opus/volltexte/2015/4921>. <https://doi.org/10.4230/LIPIcs.STACS.2015.288>
- [7] O. Reingold, Undirected ST-connectivity in log-space, *Electron. Colloq. Comput. Complex. (ECCC)* (2004). <https://doi.org/10.1145/1060590.1060647>
- [8] R.J. Lipton, R.E. Tarjan, A separator theorem for planar graphs, *J. Appl. Math. (SIAM)* 36 (2) (1979) 177–189. <https://doi.org/10.1137/0136016>
- [9] G.N. Frederickson, Fast algorithms for shortest paths in planar graphs, with applications, *SIAM J. Comput.* 16 (6) (1987) 1004–1022. <https://doi.org/10.1137/0216064>
- [10] G.L. Miller, S.-H. Teng, W. Thurston, S.A. Vavasis, Separators for sphere-packings and nearest neighbor graphs, *J. ACM* 44 (1) (1997) 1–29. <https://doi.org/10.1145/256292.256294>
- [11] K.-i. Kawarabayashi, B. Reed, A separator theorem in minor-closed classes, in: Proceedings of the 51st Annual Symposium on Foundations of Computer Science (FOCS 2010), 2010, pp. 153–162. <https://doi.org/10.1109/FOCS.2010.22>
- [12] T. Leighton, S. Rao, An approximate max-flow min-cut theorem for uniform multicommodity flow problems with applications to approximation algorithms, in: Proceedings of the 29th Annual Symposium on Foundations of Computer Science (FOCS 1988), 1988, pp. 422–431. <https://doi.org/10.1109/SFCS.1988.21958>
- [13] G.E. Blelloch, A. Farzan, Succinct representations of separable graphs, in: Proceedings of the 21st Annual Conference on Combinatorial Pattern Matching (CPM 2010), 2010, p. 138–150.
- [14] J. Hopcroft, R. Tarjan, Efficient planarity testing, *J. ACM* 21 (4) (1974) 549–568. <https://doi.org/10.1145/321850.321852>
- [15] P.N. Klein, S. Mozes, C. Sommer, Structured recursive separator decompositions for planar graphs in linear time, in: Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC 2013), Association for Computing Machinery, New York, NY, USA, 2013, p. 505–514. <https://doi.org/10.1145/2488608.2488672>
- [16] M. Elberfeld, K.-i. Kawarabayashi, Embedding and canonizing graphs of bounded genus in logspace, in: Proceedings of the 46th Annual ACM Symposium on Theory of Computing (STOC 2014), Association for Computing Machinery, 2014, p. 383–392. <https://doi.org/10.1145/2591796.2591865>

- [17] E. Allender, M. Mahajan, The complexity of planarity testing, *Inf. Comput.* 189 (1) (2004) 117–134. <https://doi.org/10.1016/j.ic.2003.09.002>
- [18] K. Keeler, J. Westbrook, Short encodings of planar graphs and maps, *Discrete Appl. Math.* 58 (3) (1995) 239–252. [https://doi.org/10.1016/0166-218X\(93\)E0150-W](https://doi.org/10.1016/0166-218X(93)E0150-W)
- [19] X. He, M. Kao, H. Lu, A fast general methodology for information-theoretically optimal encodings of graphs, *SIAM J. Comput.* 30 (3) (2000) 838–846. <https://doi.org/10.1137/S0097539799359117>
- [20] J.I. Munro, V. Raman, Succinct representation of balanced parentheses and static trees, *SIAM J. Comput.* 31 (3) (2001) 762–776. <https://doi.org/10.1137/S0097539799364092>
- [21] Y.-T. Chiang, C.-C. Lin, H.-I. Lu, Orderly spanning trees with applications to graph encoding and graph drawing, in: *Proceedings of the 12th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2001)*, Society for Industrial and Applied Mathematics, 2001, p. 506–515.
- [22] D.K. Blandford, G.E. Blelloch, I.A. Kash, Compact representations of separable graphs, in: *Proceedings of the 14th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2003)*, SIAM, 2003, p. 679–688.
- [23] M.T. Goodrich, Planar separators and parallel polygon triangulation, *J. Comput. Syst. Sci.* 51 (3) (1995) 374–389. <https://doi.org/10.1006/jcss.1995.1076>
- [24] L.C. Aleari, O. Devillers, G. Schaeffer, Succinct representation of triangulations with a boundary, in: *Algorithms and Data Structures*, Springer, 2005, pp. 134–145.
- [25] L.C. Aleari, O. Devillers, G. Schaeffer, Dynamic updates of succinct triangulations, in: *Proceedings of the 17th Canadian Conference on Computational Geometry (CCCG 2005)*, 2005, pp. 134–137.
- [26] L.C. Aleari, O. Devillers, Array-based compact data structures for triangulations: practical solutions with theoretical guarantees, *J. Comput. Geom.* 9 (1) (2018) 247–289. <https://doi.org/10.20382/jocg.v9i1a8>
- [27] L.C. Aleari, O. Devillers, G. Schaeffer, Succinct representations of planar maps, *Theor. Comput. Sci.* 408 (2–3) (2008) 174–187. <https://doi.org/10.1016/j.tcs.2008.08.016>
- [28] P. Sanders, C. Schulz, Advanced multilevel node separator algorithms, in: *Experimental Algorithms*, Springer, 2016, pp. 294–309.
- [29] E. Hogan, J.R. Johnson, M. Halappanavar, Graph coarsening for path finding in cybersecurity graphs, in: *Proceedings of the 8th Annual Cyber Security and Information Intelligence Research Workshop (CSIIRW 2013)*, Association for Computing Machinery, 2013. <https://doi.org/10.1145/2459976.2459984>
- [30] G. Karypis, V. Kumar, Analysis of multilevel graph partitioning, in: *Proceedings of the 1995 ACM/IEEE Conference on Supercomputing (SC 1995)*, 1995, pp. 29–29. <https://doi.org/10.1109/SUPERC.1995.242800>
- [31] C. Chevalier, I. Saffro, Comparison of coarsening schemes for multilevel graph partitioning, in: T. Stütze (Ed.), *Proceedings of the 3rd International Conference on Learning and Intelligent Optimization (LION 2009)*, 5851 of *Lecture Notes in Computer Science*, Springer, 2009, pp. 191–205. [https://doi.org/10.1007/978-3-642-11169-3\\_14](https://doi.org/10.1007/978-3-642-11169-3_14)
- [32] M. Fahrback, G. Goranci, R. Peng, S. Sachdeva, C. Wang, Faster graph embeddings via coarsening, in: *Proceedings of the 37th International Conference on Machine Learning (ICML 2020)*, 119 of *Proceedings of Machine Learning Research*, PMLR, 2020, pp. 2953–2963. <https://proceedings.mlr.press/v119/fahrback20a.html>. <https://doi.org/10.5555/3524938.3525215>
- [33] G. Karypis, V. Kumar, A fast and high quality multilevel scheme for partitioning irregular graphs, *SIAM J. Sci. Comput.* 20 (1) (1998) 359–392.
- [34] F. Pellegrini, J. Roman, Scotch: a software package for static mapping by dual recursive bipartitioning of process and architecture graphs, in: *High-Performance Computing and Networking*, Springer, 1996, pp. 493–498.
- [35] B.F. Auer, R.H. Bisseling, A GPU algorithm for greedy graph matching, in: *Facing the Multicore-Challenge II*, Springer, 2012, pp. 108–119.
- [36] B.F. Auer, R.H. Bisseling, Graph coarsening and clustering on the GPU, *Graph Partitioning Graph Clust.* 588 (2012) 223.
- [37] C. Cai, D. Wang, Y. Wang, Graph Coarsening with Neural Networks, 2021, 2102.01350
- [38] F. Kammer, J. Meintrup, A. Sajenko, Space-efficient vertex separators for treewidth, *CoRR abs/1907.00676* (2019).
- [39] T. Izumi, Y. Otachi, Sublinear-space lexicographic depth-first search for bounded treewidth graphs and planar graphs, in: *Proceedings of the 47th International Colloquium on Automata, Languages, and Programming (ICALP 2020)*, 168 of *Leibniz International Proceedings in Informatics (LIPIcs)*, Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2020, pp. 67:1–67:17. <https://doi.org/10.4230/LIPIcs.ICALP.2020.67>
- [40] T. Imai, K. Nakagawa, A. Pavan, N.V. Vinodchandran, O. Watanabe, An  $O(n^{1/2+\epsilon})$ -space and polynomial-time algorithm for directed planar reachability, in: *2013 IEEE Conference on Computational Complexity, IEEE*, 2013, pp. 277–286. <https://doi.org/10.1109/CCC.2013.35>
- [41] G.N. Frederickson, Upper bounds for time-space trade-offs in sorting and selection, *J. Comput. Syst. Sci.* 34 (1) (1987) 19–26. [https://doi.org/10.1016/0022-0000\(87\)90002-X](https://doi.org/10.1016/0022-0000(87)90002-X)
- [42] R. Diestel, A. Schrijver, P.D. Seymour, *Graph Theory*, 2007.
- [43] J.R. Gilbert, J.P. Hutchinson, R.E. Tarjan, A separator theorem for graphs of bounded genus, *J. Algorithms* 5 (3) (1984) 391–407. [https://doi.org/10.1016/0196-6774\(84\)90019-1](https://doi.org/10.1016/0196-6774(84)90019-1)
- [44] R.J. Lipton, D.J. Rose, R.E. Tarjan, Generalized nested dissection, *SIAM J. Numer. Anal.* 16 (1977) 346–358.
- [45] R. Raman, V. Raman, S.R. Satti, Succinct indexable dictionaries with applications to encoding k-ary trees, prefix sums and multisets, *ACM Trans. Algorithms* 3 (4) (2007) 43–es. <https://doi.org/10.1145/1290672.1290680>
- [46] T. Hagerup, A constant-time colored choice dictionary with almost robust iteration, in: *Proceedings of the 44th International Symposium on Mathematical Foundations of Computer Science (MFCS 2019)*, 138 of *LIPIcs*, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019, pp. 64:1–64:14. <https://doi.org/10.4230/LIPIcs.MFCS.2019.64>
- [47] F. Kammer, A. Sajenko, Simple  $2^l$ -color choice dictionaries, in: *Proceedings of the 29th International Symposium on Algorithms and Computation (ISAAC 2018)*, 123 of *Leibniz International Proceedings in Informatics (LIPIcs)*, Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2018, pp. 66:1–66:12. <https://doi.org/10.4230/LIPIcs.ISAAC.2018.66>
- [48] F. Kammer, D. Kratsch, M. Laudahn, Space-efficient biconnected components and recognition of outerplanar graphs, *Algorithmica* 81 (3) (2019) 1180–1204. <https://doi.org/10.1007/s00453-018-0464-z>
- [49] N. Banerjee, S. Chakraborty, V. Raman, et al., Improved space efficient algorithms for BFS, DFS and applications, in: *Computing and Combinatorics*, Springer, 2016, pp. 119–130.
- [50] T. Hagerup, F. Kammer, M. Laudahn, Space-efficient Euler partition and bipartite edge coloring, *Theor. Comput. Sci.* 754 (2019) 16–34. <https://doi.org/10.1016/j.tcs.2018.01.008>
- [51] G.L. Miller, Finding small simple cycle separators for 2-connected planar graphs, *J. Comput. Syst. Sci.* 32 (3) (1986) 265–279. [https://doi.org/10.1016/0022-0000\(86\)90030-9](https://doi.org/10.1016/0022-0000(86)90030-9)
- [52] P. Klein, S. Mozes, Separators in planar graphs, 2021, [http://planarity.org/Klein\\_separators\\_in\\_planar\\_graphs.pdf](http://planarity.org/Klein_separators_in_planar_graphs.pdf).